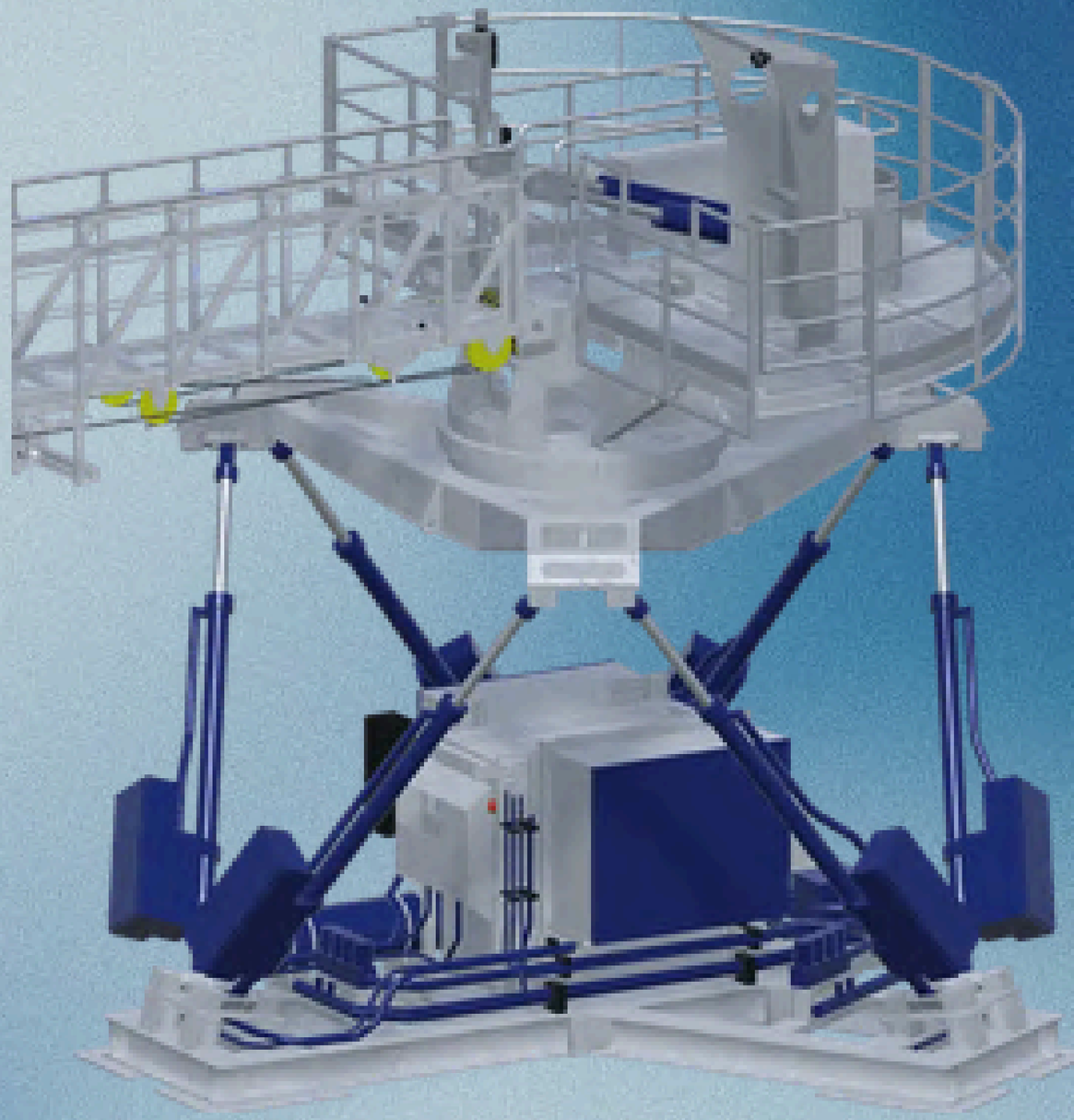
Stabilisation d’une plateforme de Gough-Stewart pour la compensation de la houle

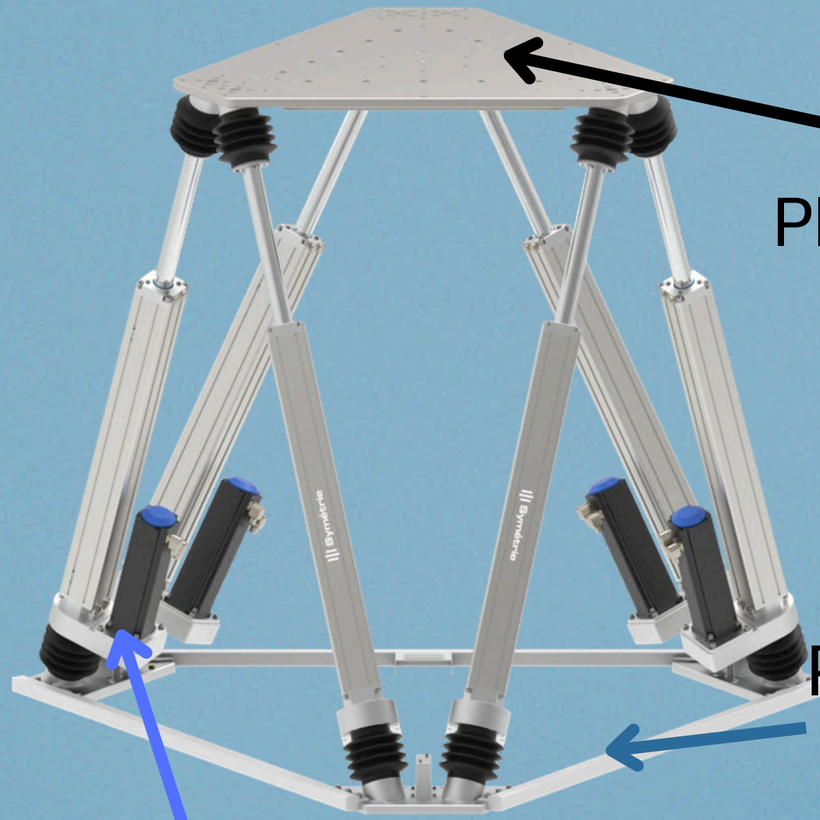


N°SCEI : 12885

Bilal BELKHAOUEL

TSI 2026

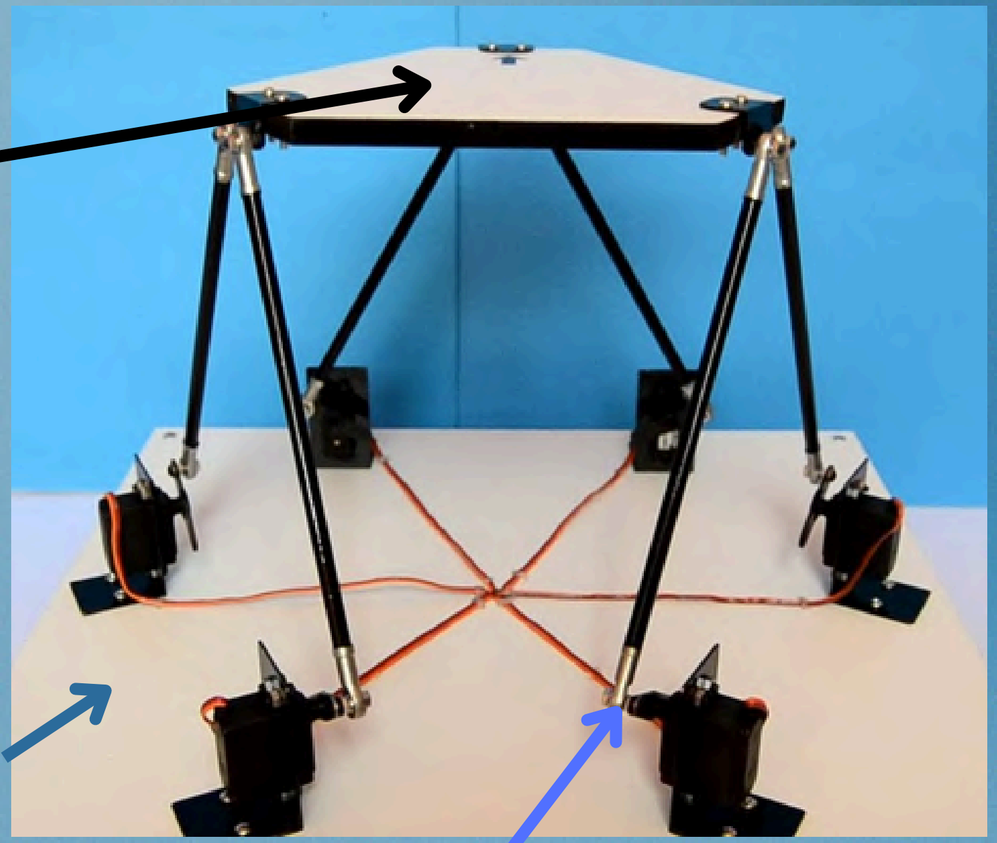
Qu’est ce qu’une plateforme de Gough-Stewart ?



Plateforme supérieure
(à stabiliser)

Plateforme inférieure
(fixée au navire)

6 vérins



6 systèmes pseudo-bielle manivelle
(vérins)

Utilisation d’une plateforme de Gough-Stewart



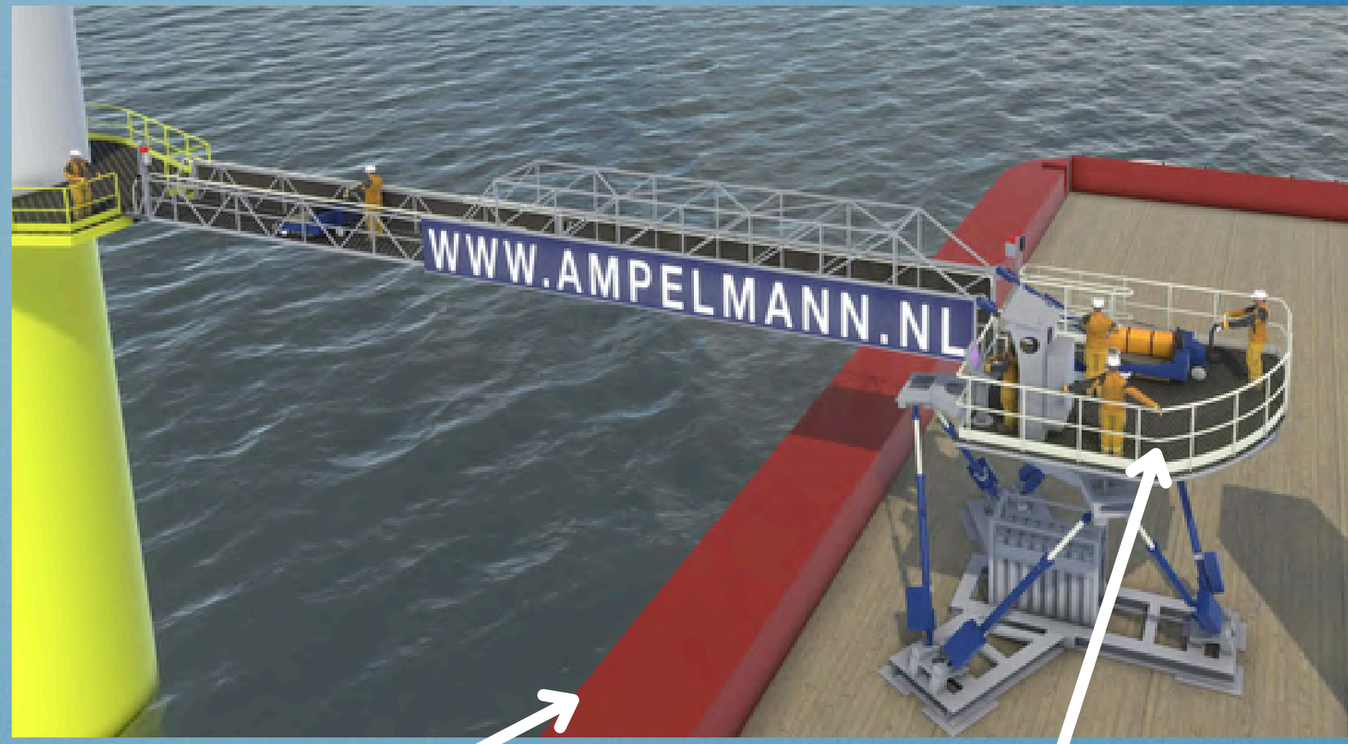




Notre cas d’utilisation de la plateforme de Gough-Stewart

Personnel et matériel

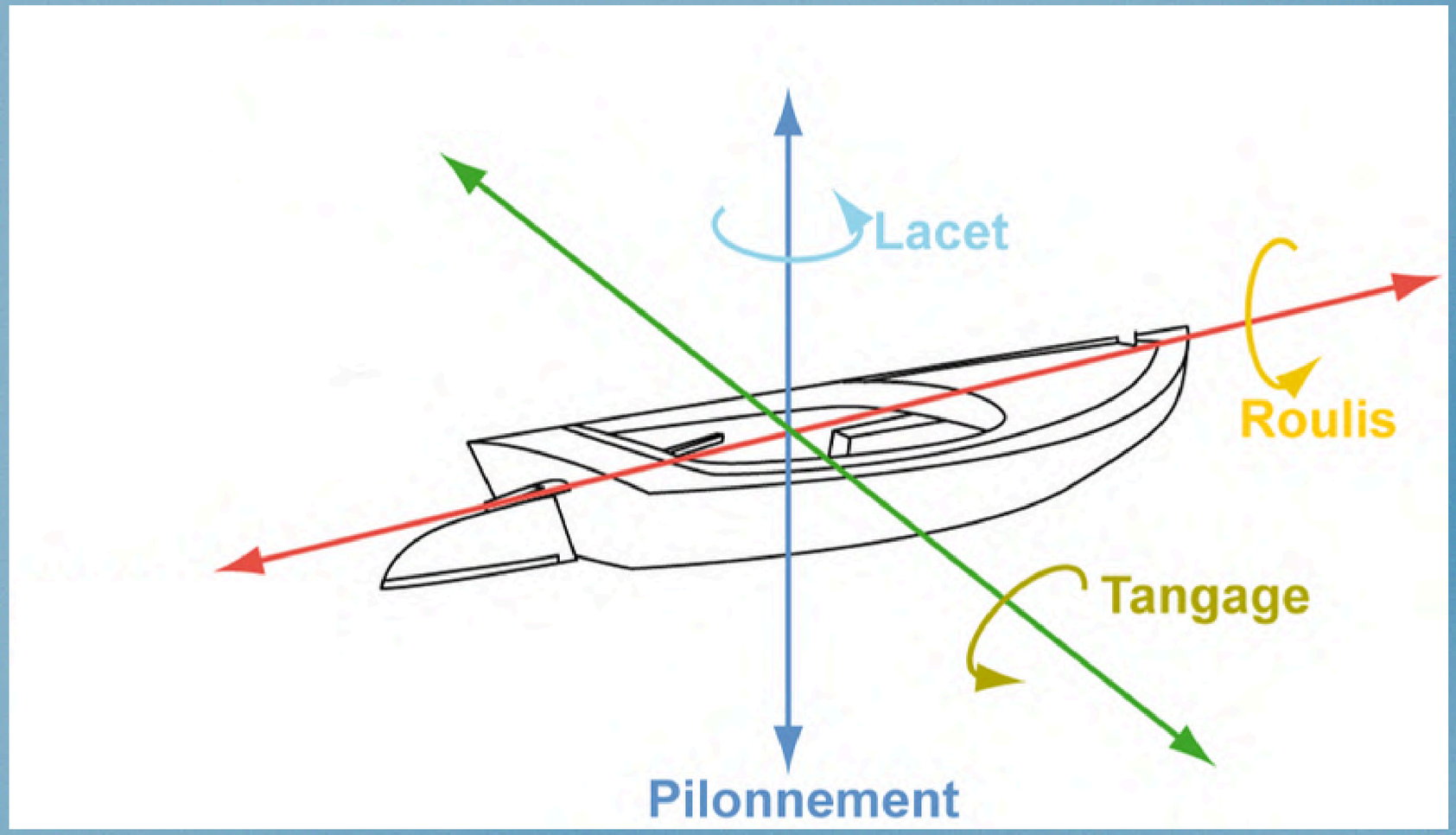




Navire soumis à la houle

Plateforme à stabiliser

Notre cas d’utilisation de la plateforme de Gough-Stewart



Le but est de stabiliser les axes de roulis et tangage

Sommaire

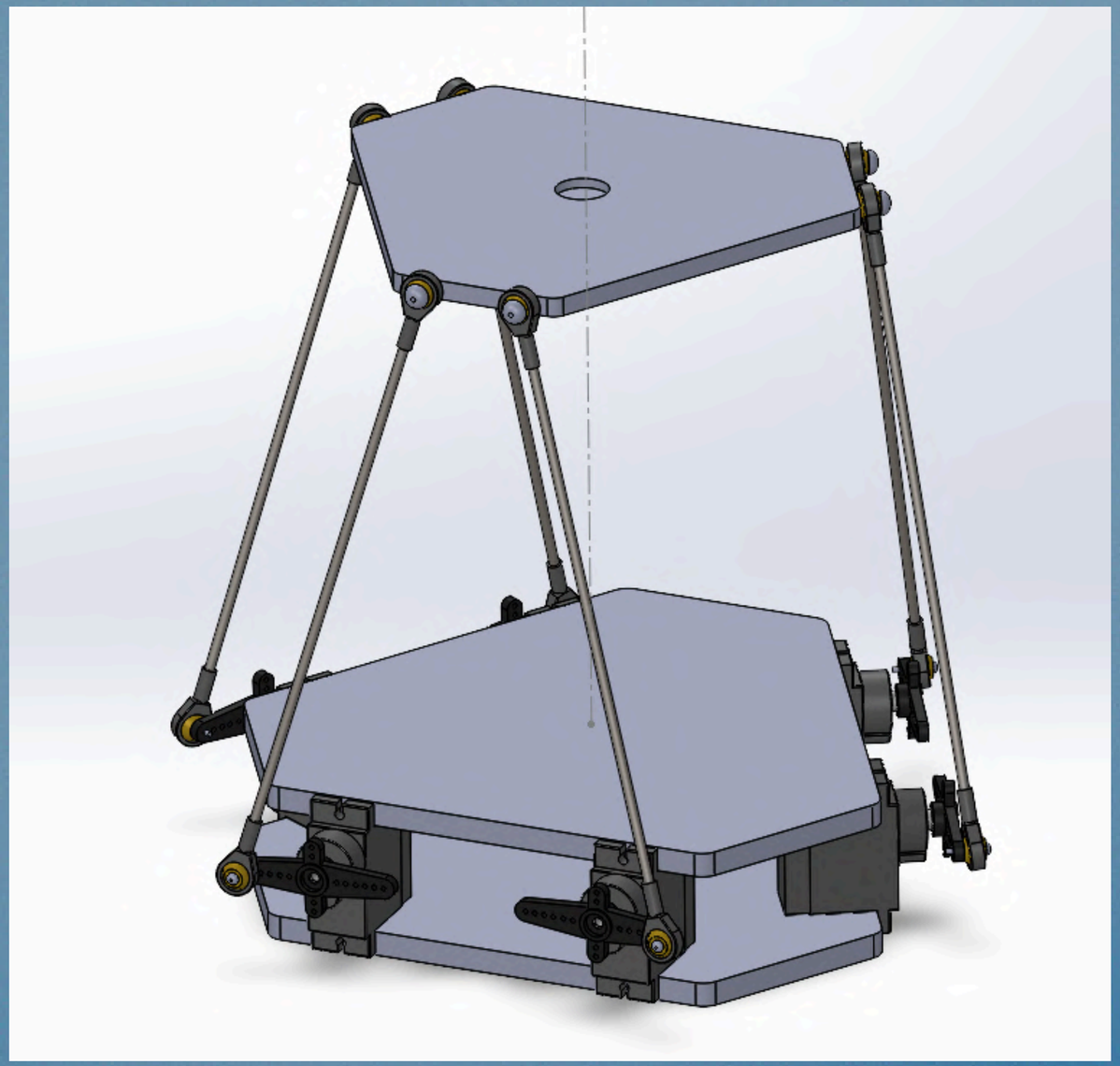
I- Modélisation et analyse du système :

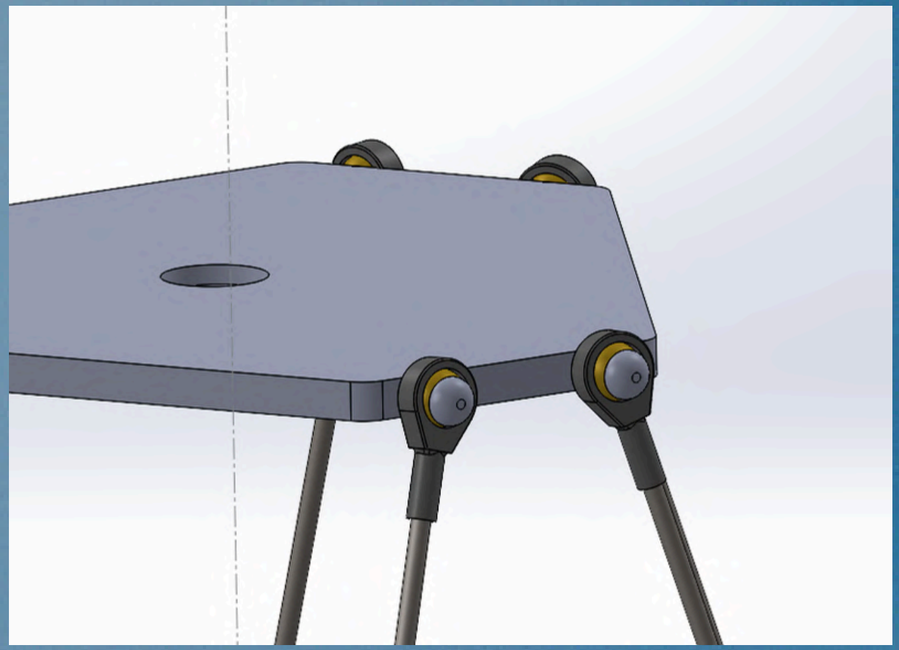
- Modèle étudié
- Schéma pour la mise en œuvre
- Fonctions fondamentales
- Relation angle servomoteur/plateau supérieur
- Mesure d’angle
- Filtre complémentaire

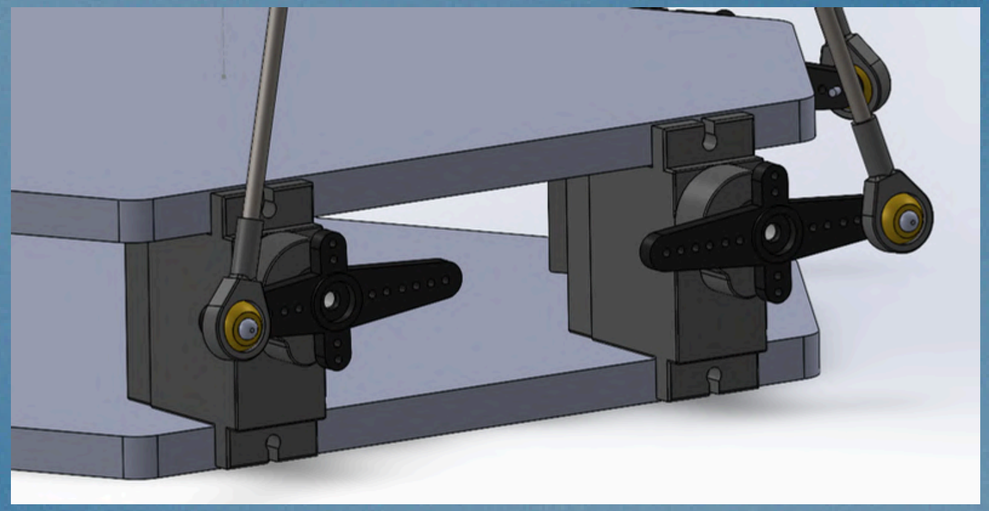
II- Mise en œuvre :

- Boucle ouverte
- Boucle fermée

Maquette 3D



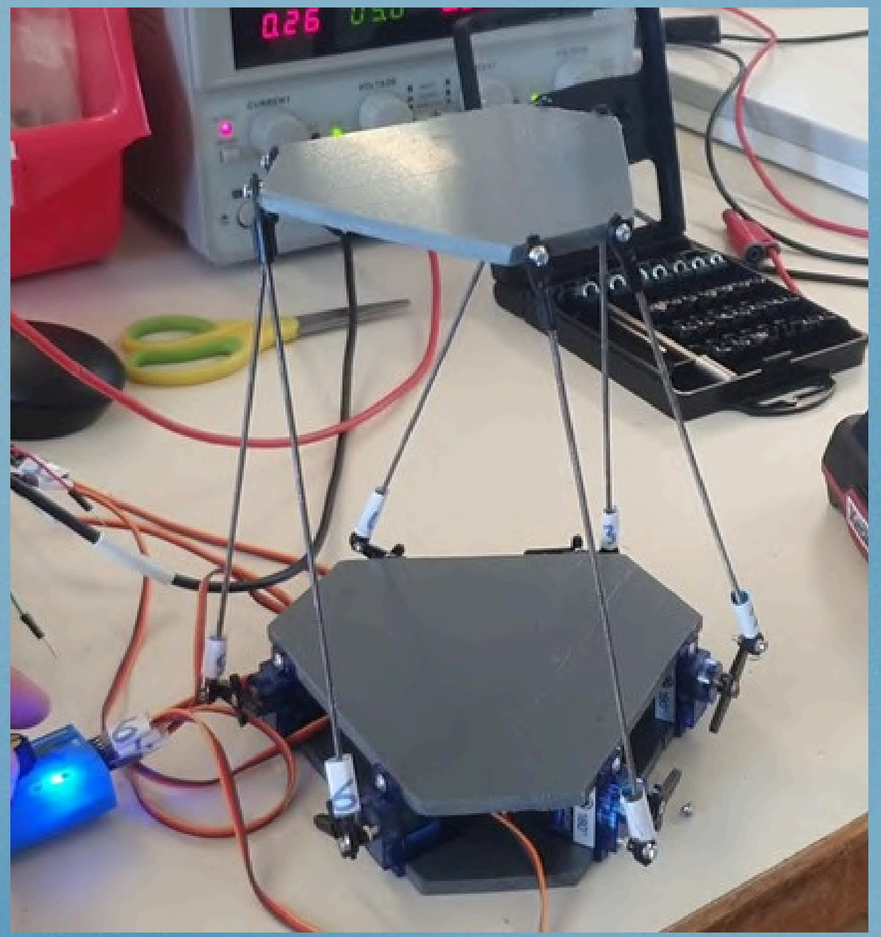




Maquette réelle







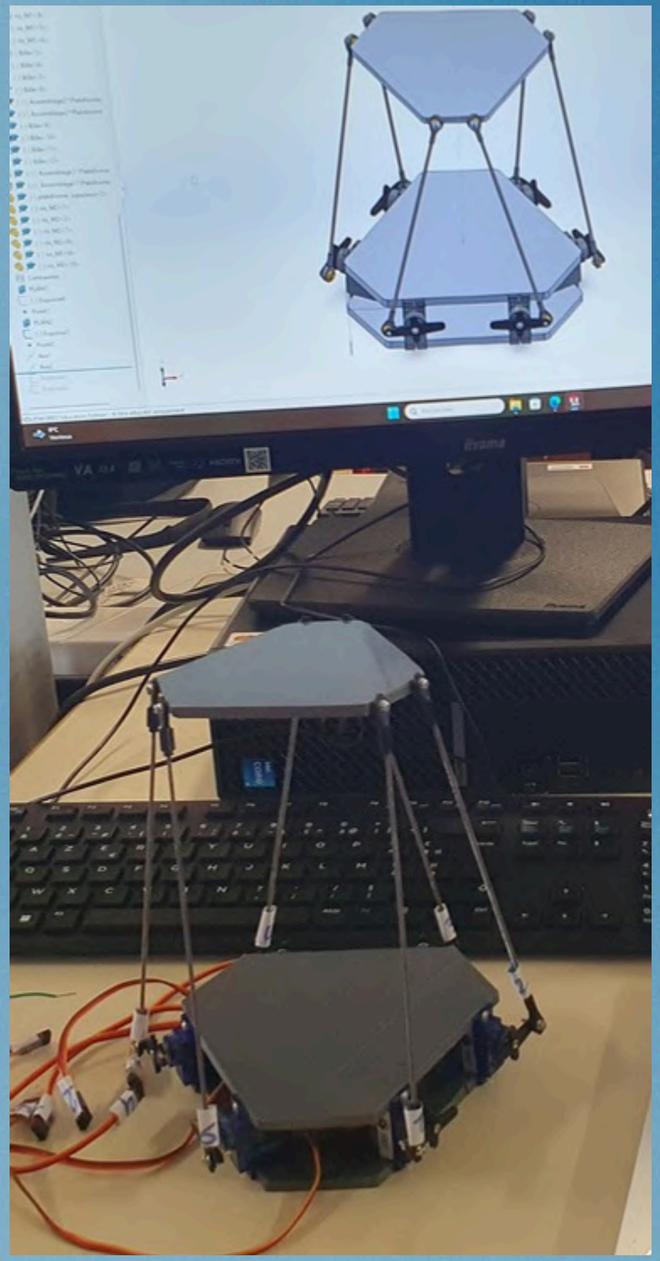
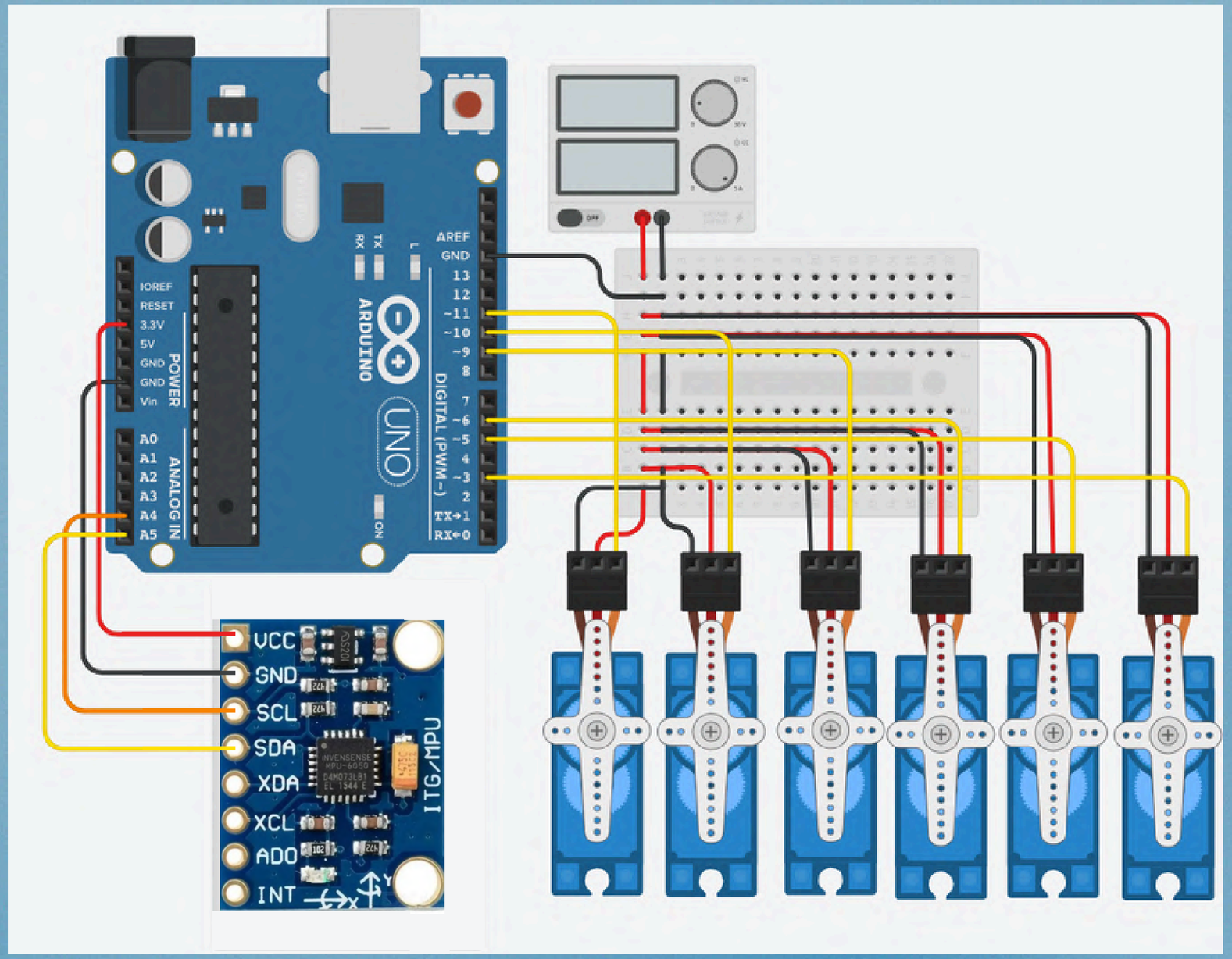
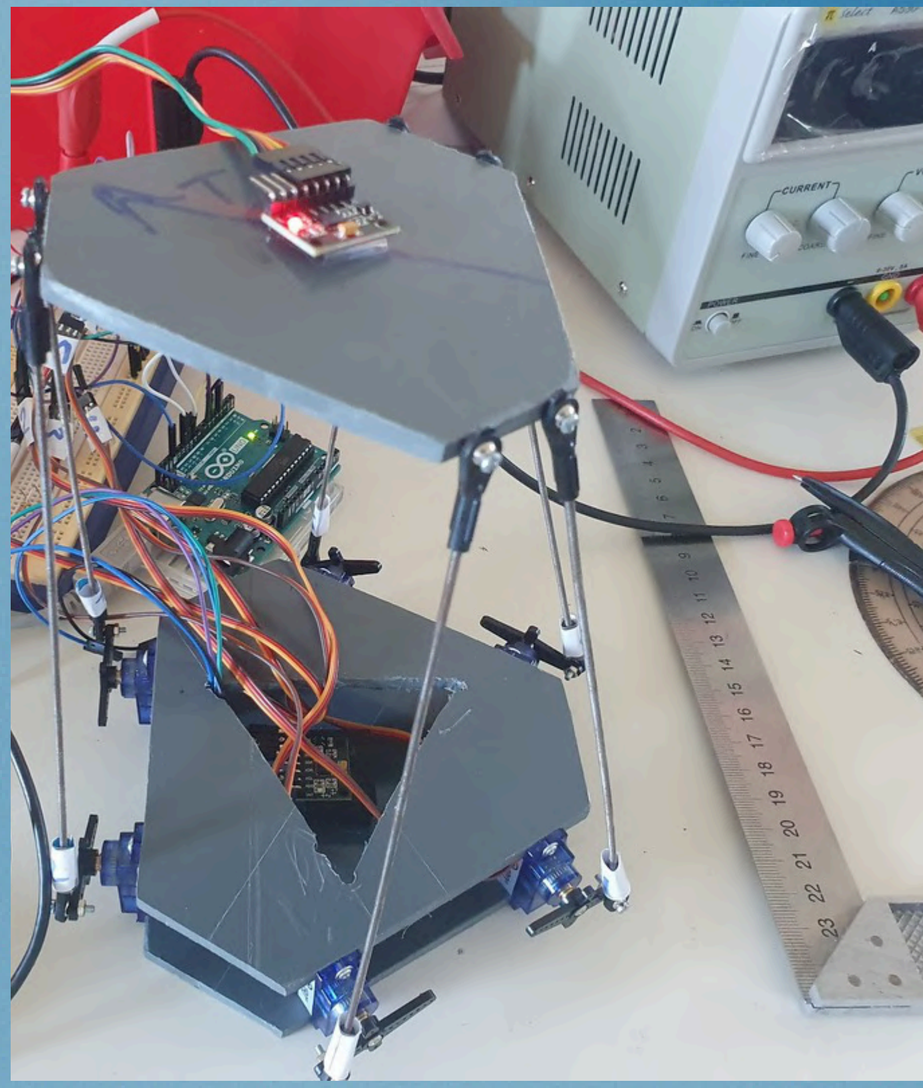


Schéma pour la mise en œuvre





Stratégie de commande

Boucle ouverte

→Commande de la plateforme : en fonction de **l'inclinaison du navire**

→ On mesure l'angle de la plateforme **inférieure**

→ Pas de retour de ce qu'il se passe en haut

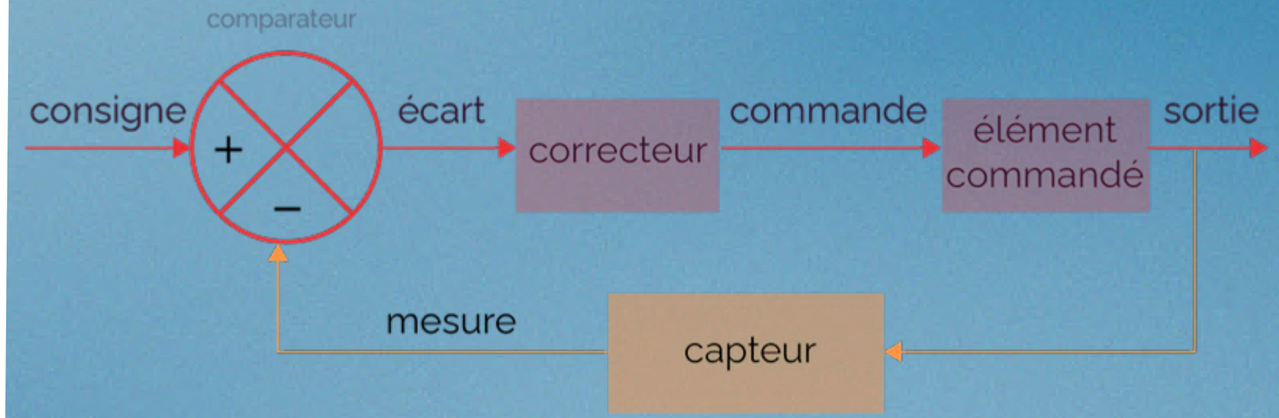


Boucle fermée

→ On mesure l'inclinaison de la **plateforme supérieure**

→ On corrige au fur et à mesure

→ Consigne de 0°



Modèle géométrique direct et indirect

MGD

On calcule la position de la plateforme supérieure

$$\theta = f(A_x)$$

MGI

On calcule la position des servomoteurs

$$A_x = f^{-1}(\theta)$$

A_x : Amplitude angulaire des servomoteurs

θ : Angle du plateau supérieur

Fonctions fondamentales et fonction principale

Les fonctions fondamentales

Qualitatif

$$V_x = \begin{pmatrix} \pm 1 \\ \pm 1 \\ \pm 1 \\ \pm 1 \\ \pm 1 \\ \pm 1 \end{pmatrix}$$

La fonction principale

Quantitatif

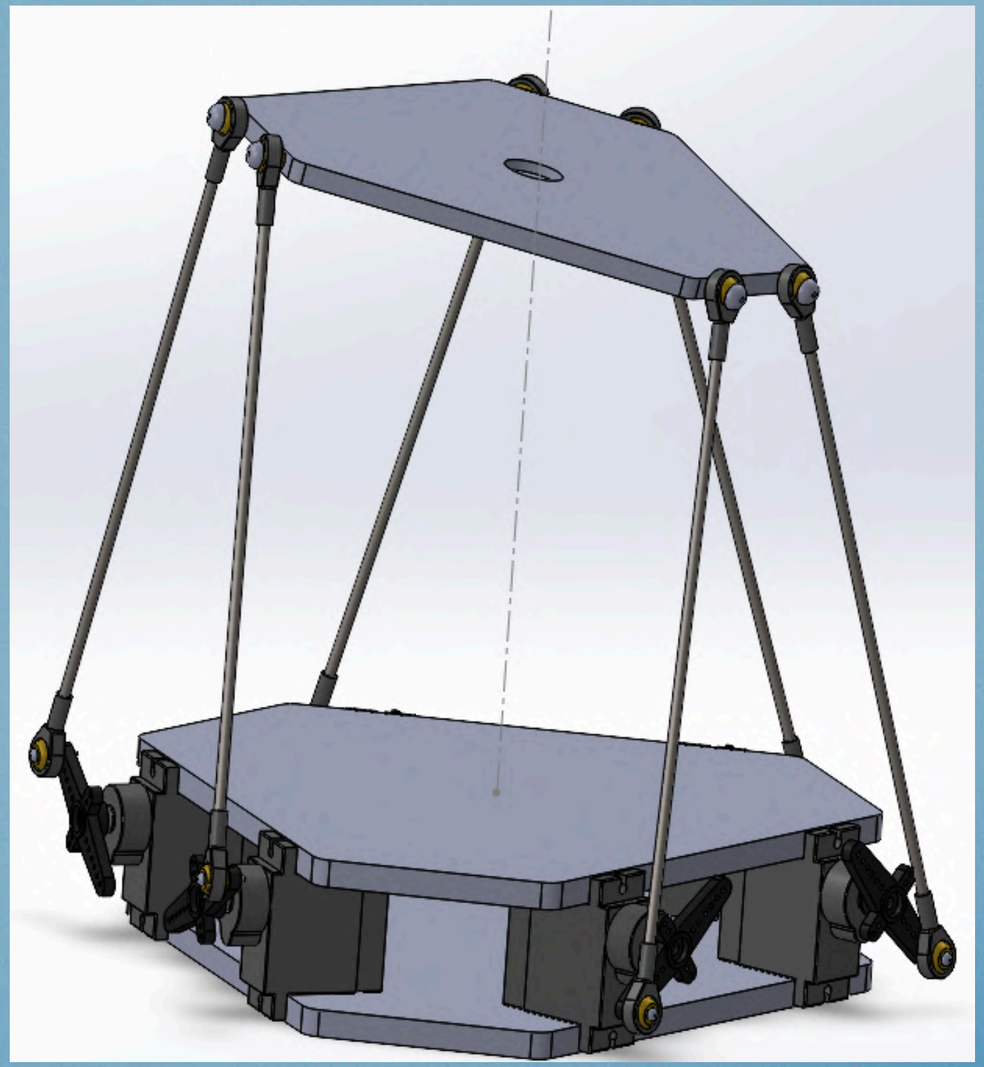
On peut contrôler uniquement l'angle des servomoteurs :

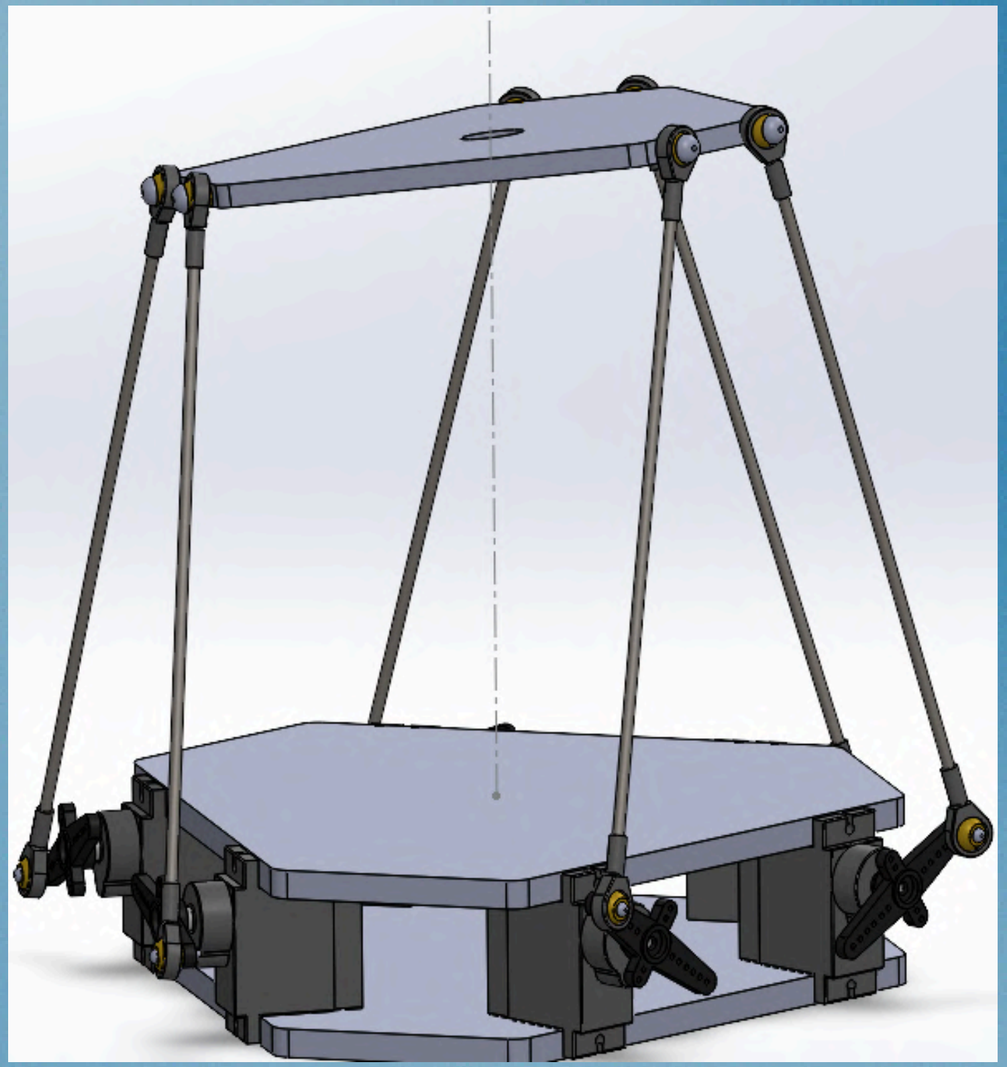
$$\alpha(t) = \alpha_{neutre} + A_x(t) \cdot V_x$$

$$\alpha(t) = \begin{pmatrix} \alpha_1 \\ \alpha_2 \\ \alpha_3 \\ \alpha_4 \\ \alpha_5 \\ \alpha_6 \end{pmatrix} \qquad \alpha_{neutre} = \begin{pmatrix} 90 \\ 90 \\ 90 \\ 90 \\ 90 \\ 90 \end{pmatrix}$$

Trouvons les fonctions fondamentales :

Utilisation de Meca3D dans SolidWorks



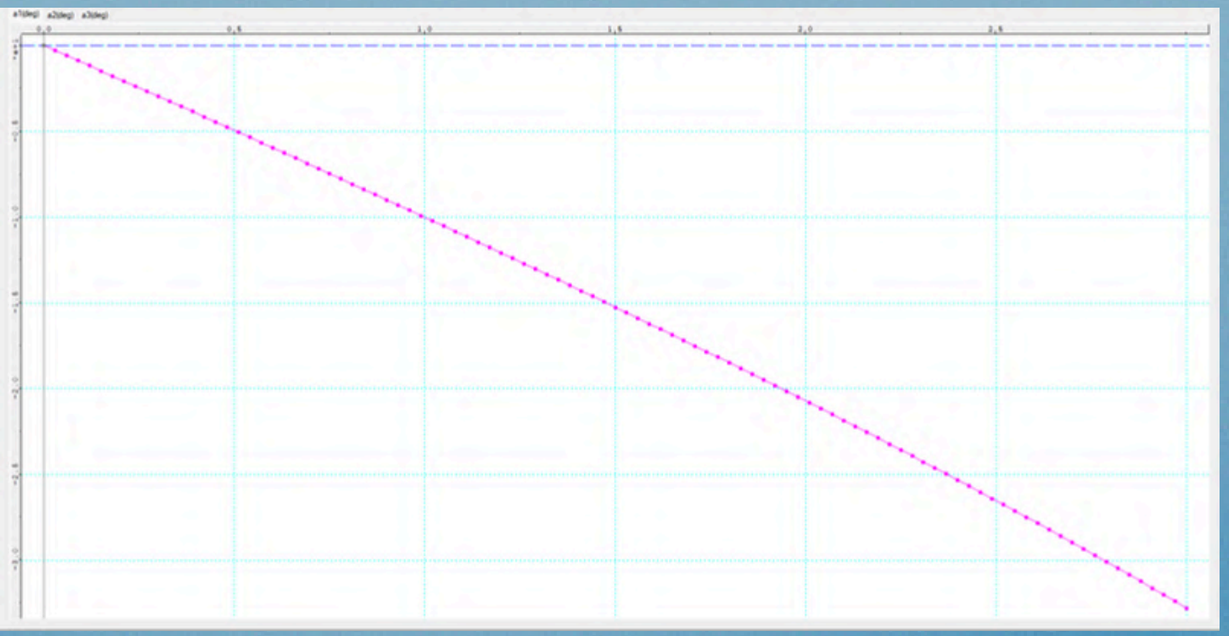


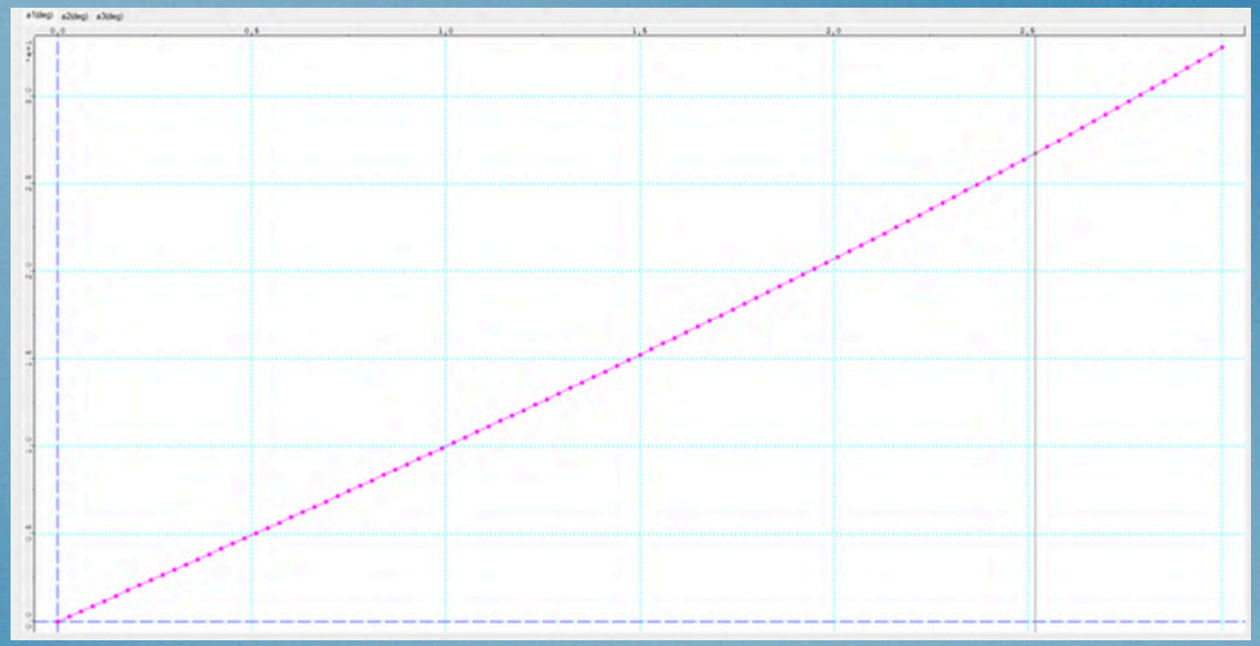
Utilisation de Meca3D dans SolidWorks

Scenario 1

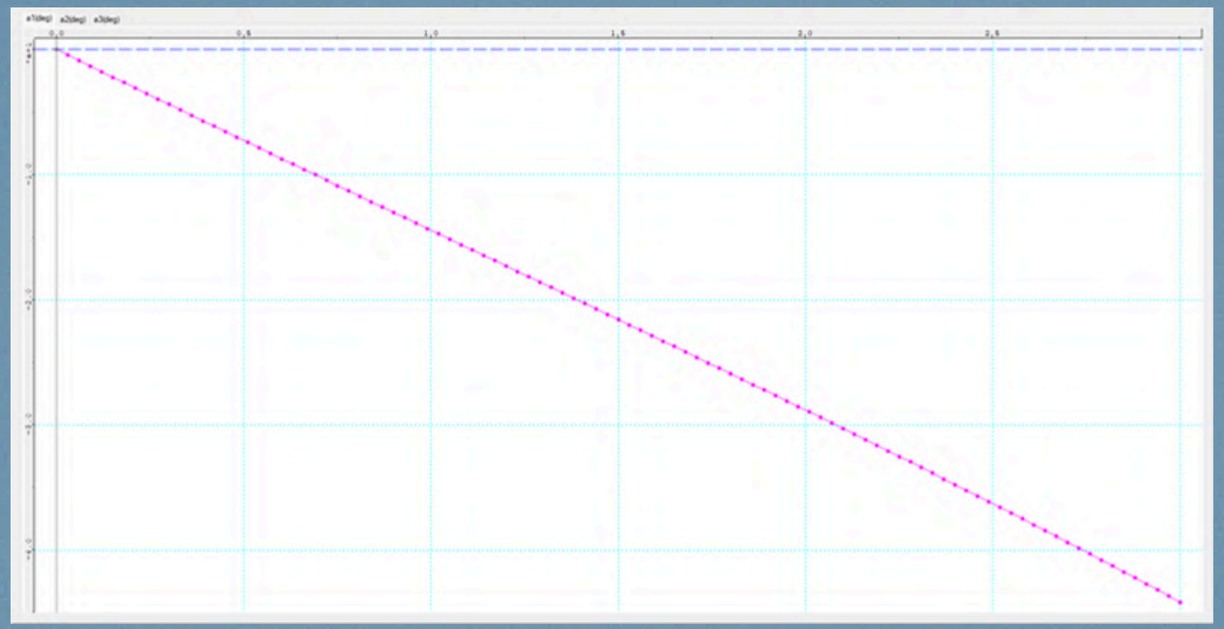
N°	Liaison	Compos.	Mouvement	Vitesse	Entrée
1	Généralisée1	Rx (0.00000...	Uniforme	0.000000	
2	Généralisée1	Ry (-8.6602...	Uniforme	0.000000	
3	Généralisée1	Rz (5.00000...	Uniforme	0.666667	
4	Généralisée1	Tx (0.00000...	Uniforme	0.000000	
5	Généralisée1	Ty (-8.6602...	Uniforme	0.000000	

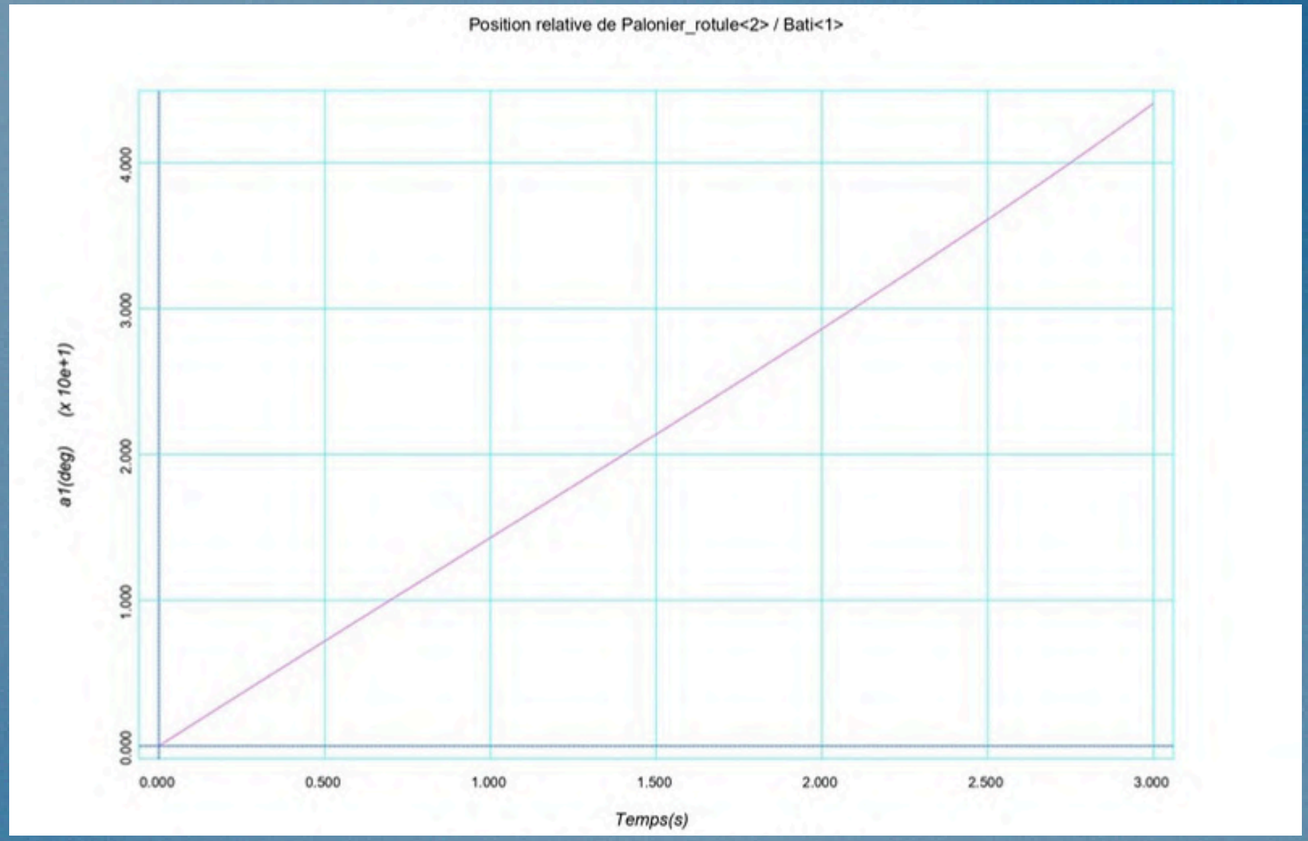
⅔ tr/min sur 3s de simulation -> 12°





Utilisation de Meca3D dans SolidWorks





Utilisation de Meca3D dans SolidWorks

Actionneur	Roulis (X > 0)	Tangage (Y > 0)
Servomoteur 1	Horaire	Trigonométrique
Servomoteur 2	Trigonométrique	Trigonométrique
Servomoteur 3	Trigonométrique	Trigonométrique
Servomoteur 4	Horaire	Horaire
Servomoteur 5	Trigonométrique	Horaire
Servomoteur 6	Horaire	Horaire

Utilisation de Meca3D dans SolidWorks

Actionneur	Trans. X	Trans. Y	Trans. Z	Rot. X (Roulis)	Rot. Y (Tangage)	Rot. Z (Lacet)
Servomoteur 1	Horaire	Horaire	Horaire	Horaire	Trigo	Horaire
Servomoteur 2	Horaire	Trigo	Trigo	Trigo	Trigo	Horaire
Servomoteur 3	Trigo	Configuration particulière	Horaire	Trigo	Trigo	Horaire
Servomoteur 4	Trigo	Horaire	Trigo	Horaire	Horaire	Horaire
Servomoteur 5	Horaire	Configuration particulière	Horaire	Trigo	Horaire	Horaire
Servomoteur 6	Horaire	Trigo	Trigo	Horaire	Horaire	Horaire

Trouvons maintenant la fonction principale :

Relation angles servomoteur/plateau supérieur

De manière purement mathématique :

$$\mathbf{R} = \begin{pmatrix} \cos\psi & -\sin\psi & 0 \\ \sin\psi & \cos\psi & 0 \\ 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} \cos\theta & 0 & \sin\theta \\ 0 & 1 & 0 \\ -\sin\theta & 0 & \cos\theta \end{pmatrix} \cdot \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos\varphi & -\sin\varphi \\ 0 & \sin\varphi & \cos\varphi \end{pmatrix} \dots$$

...

$$\alpha_i(\theta) = \arcsin\left(\frac{l_i^2(\theta) - s^2 + a^2}{\sqrt{M^2 + N^2}}\right) - \arctan\left(\frac{N}{M}\right)$$

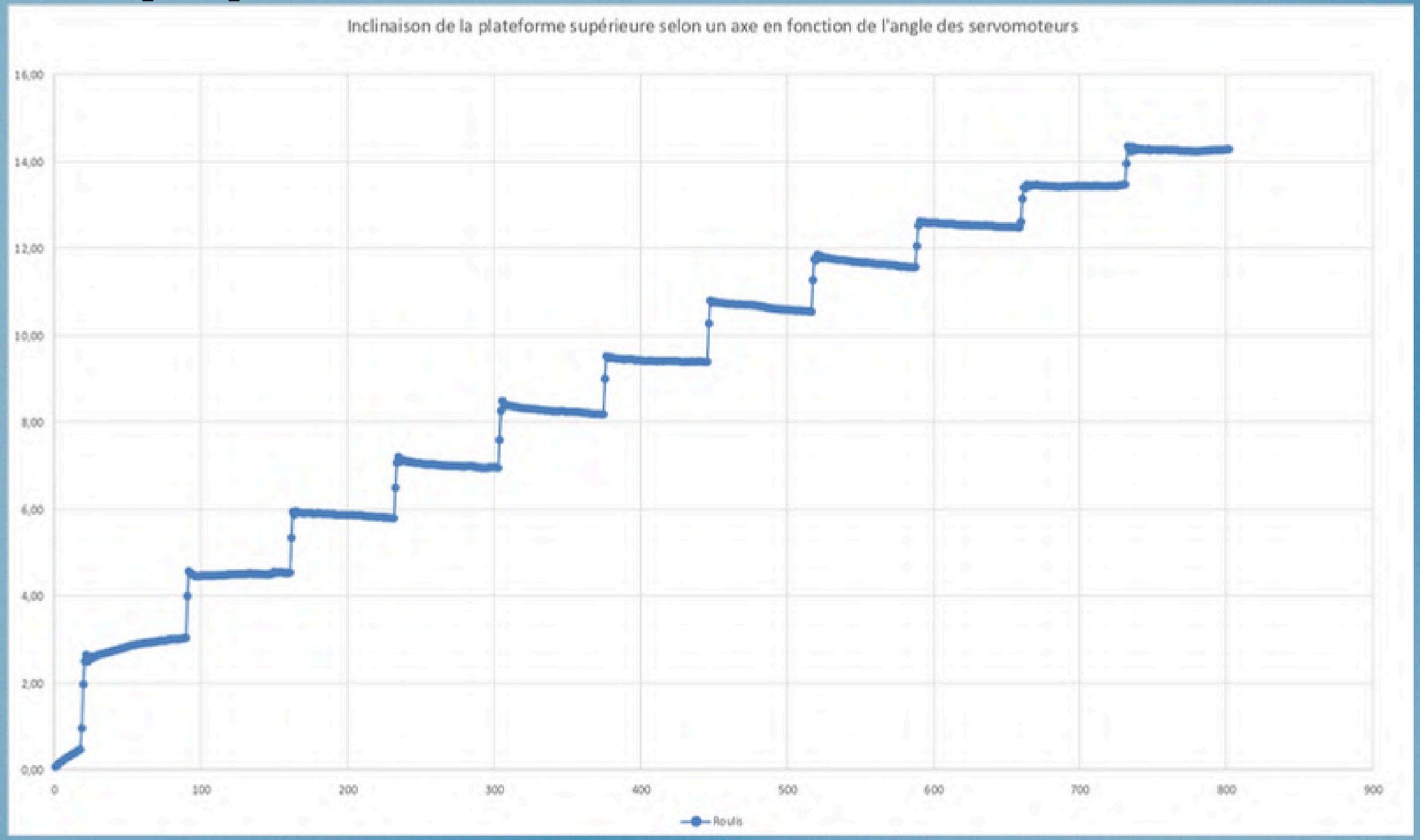
...

$$f^{-1}(\theta) = |\alpha_i(\theta) - \alpha_{neutre}|$$

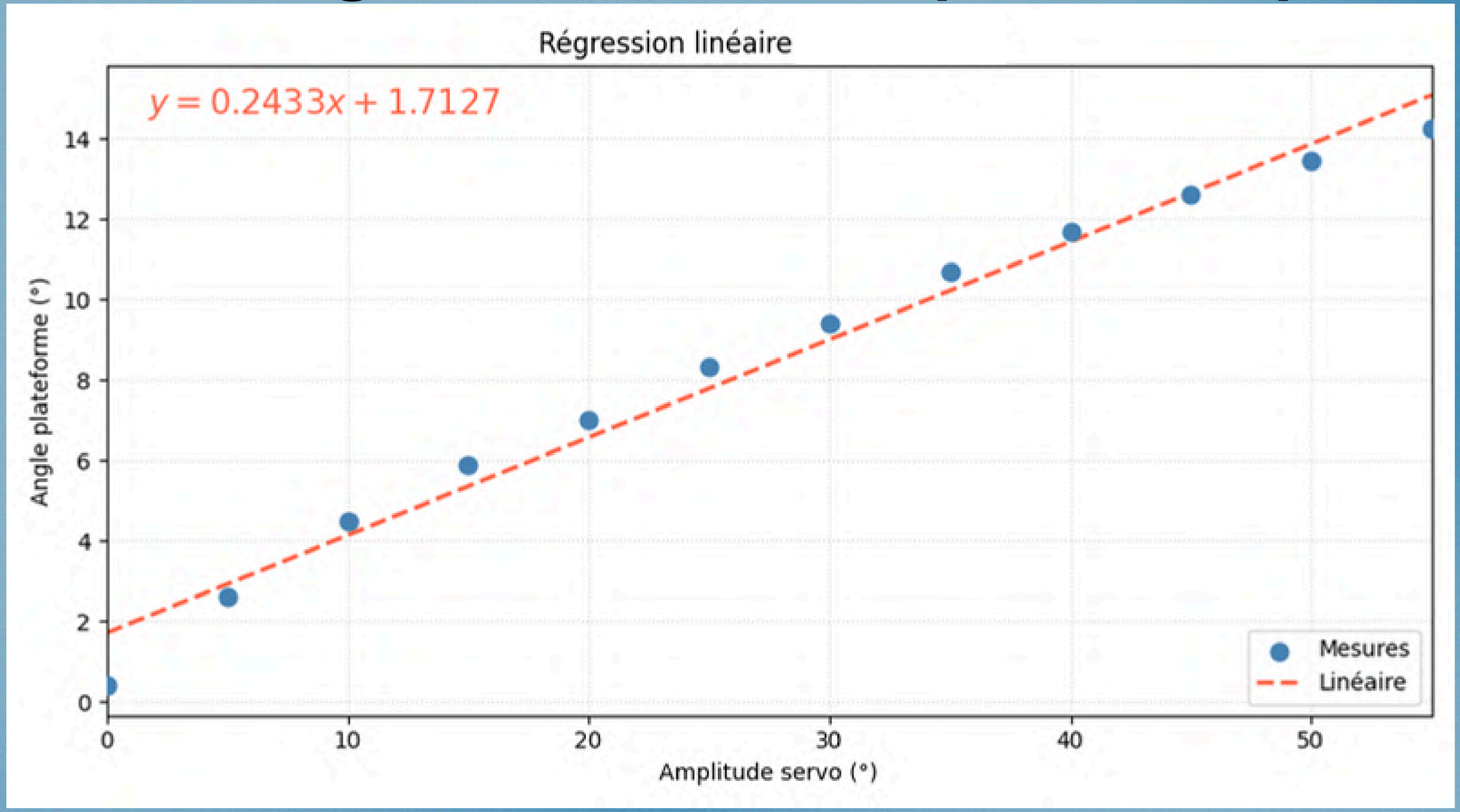
$$\begin{cases} \psi : \text{angle de lacet} \\ \theta : \text{angle de tangage} \\ \varphi : \text{angle de roulis} \end{cases}$$

Relation angles servomoteur/plateau supérieur

De manière empirique :

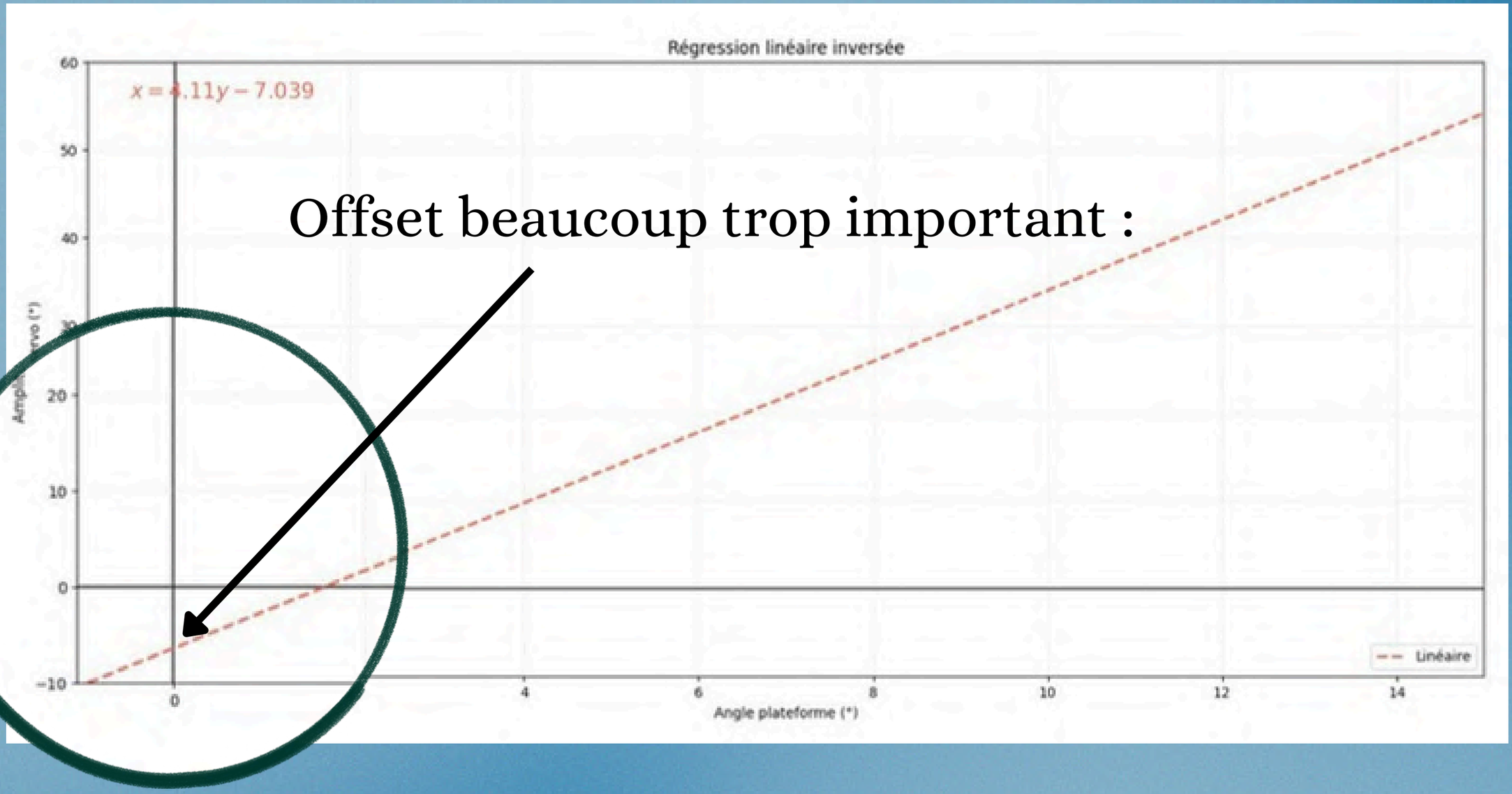


Relation angles servomoteur/plateau supérieur

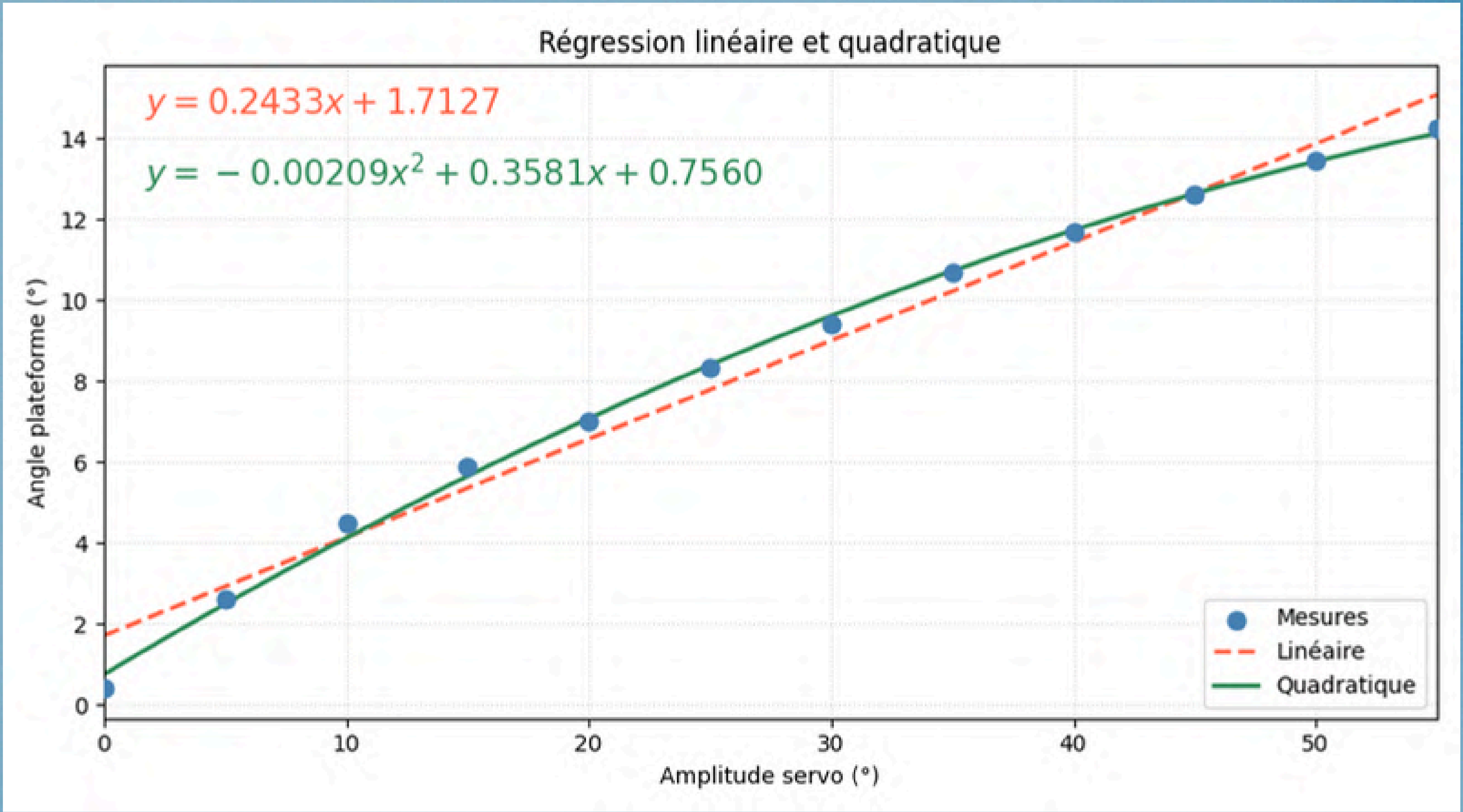


Relation angles servomoteur/plateau supérieur

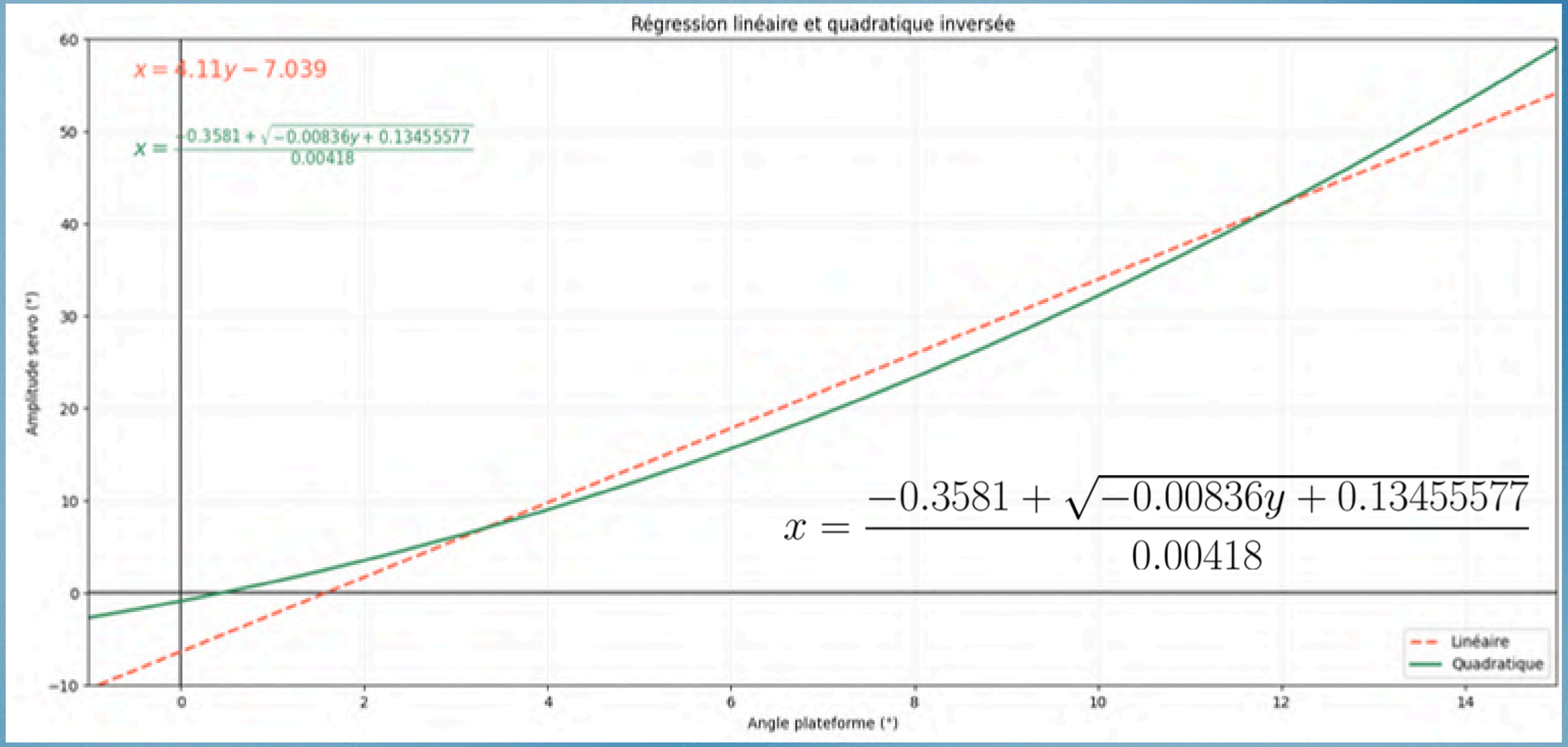
Cependant :



Relation angles servomoteur/plateau supérieur



Relation angles servomoteur/plateau supérieur

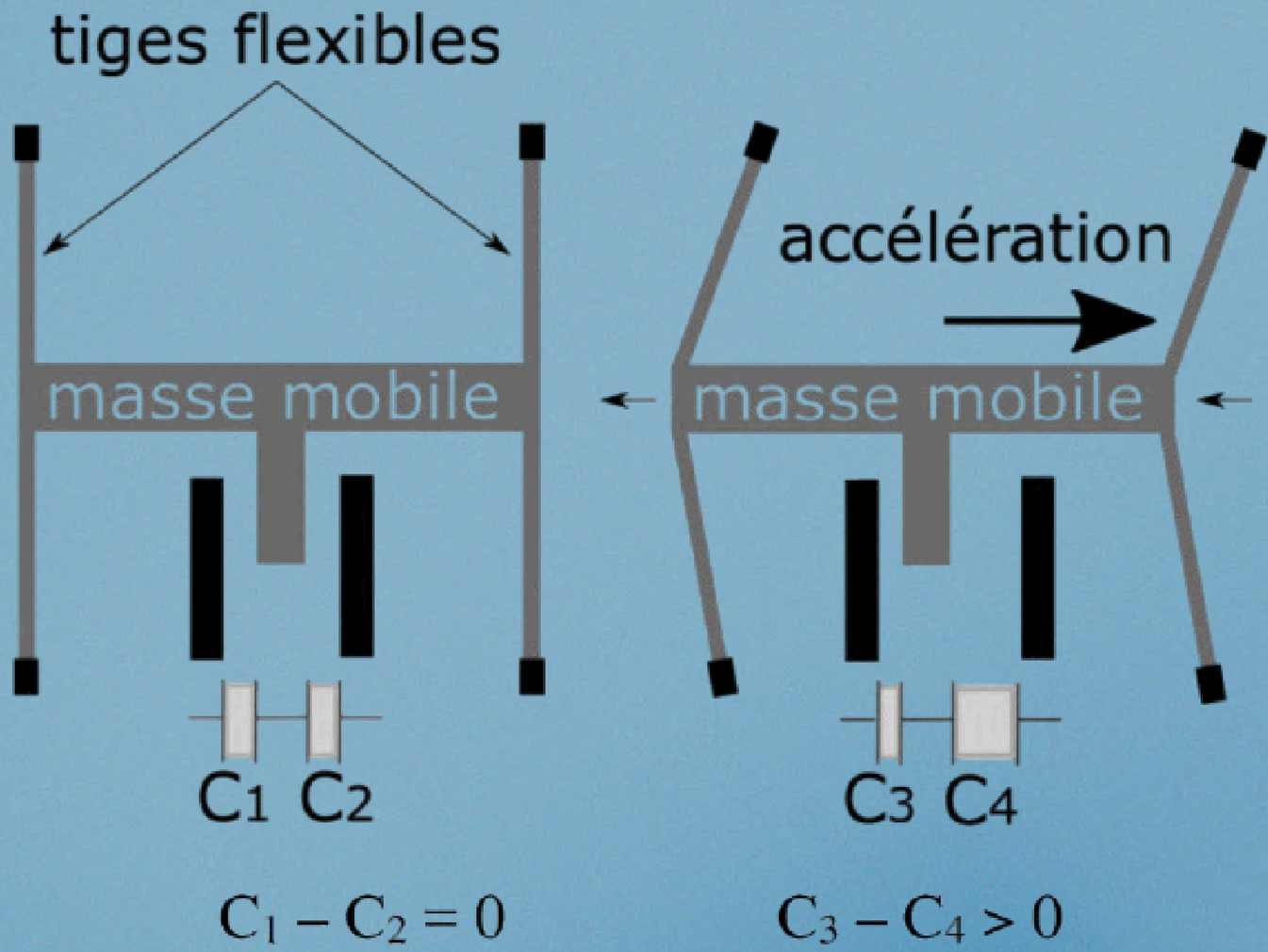


Relation angles servomoteur/plateau supérieur

$$\alpha(t) = \alpha_{neutre} + A_x(t) \cdot V_x$$

$$\alpha(t) = \begin{pmatrix} \alpha_1 \\ \alpha_2 \\ \alpha_3 \\ \alpha_4 \\ \alpha_5 \\ \alpha_6 \end{pmatrix} \quad \alpha_{neutre} = \begin{pmatrix} 90 \\ 90 \\ 90 \\ 90 \\ 90 \\ 90 \end{pmatrix} \quad A_x = f^{-1}(\theta) \quad V_x = \begin{pmatrix} +1 \\ -1 \\ -1 \\ +1 \\ -1 \\ +1 \end{pmatrix}$$

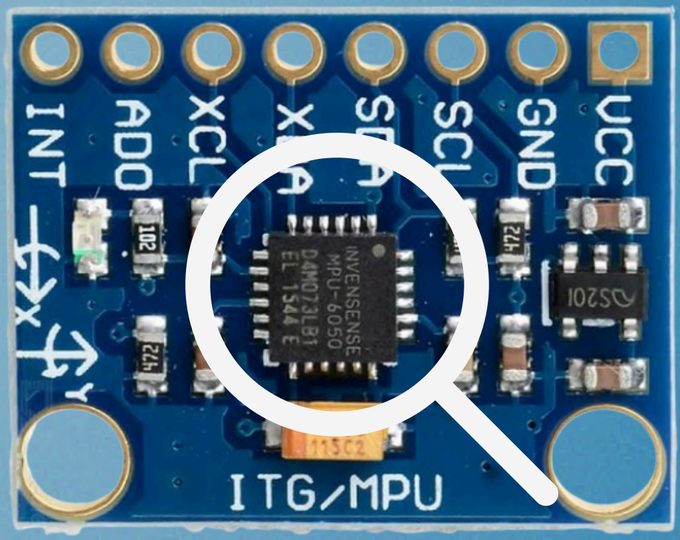
Comment fonctionne l'accéléromètre ?

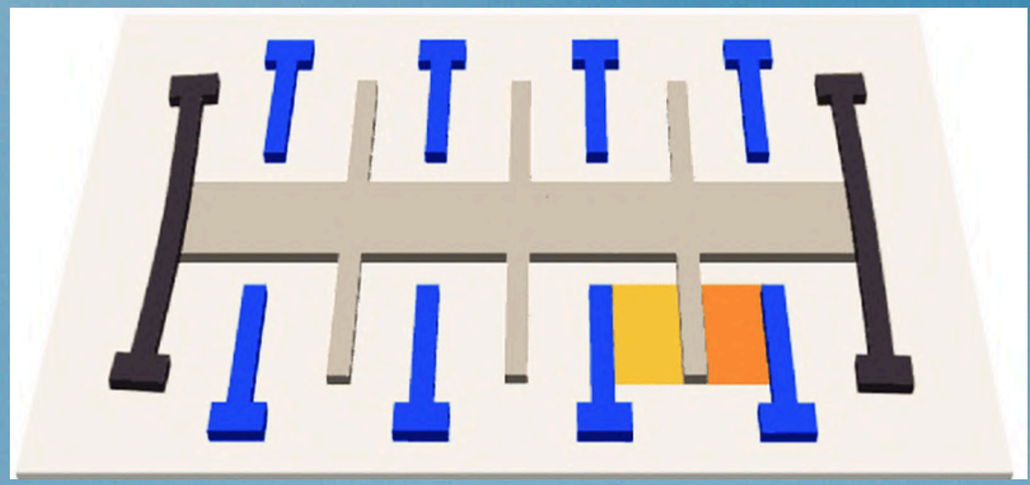


$C_1 - C_2 = 0$

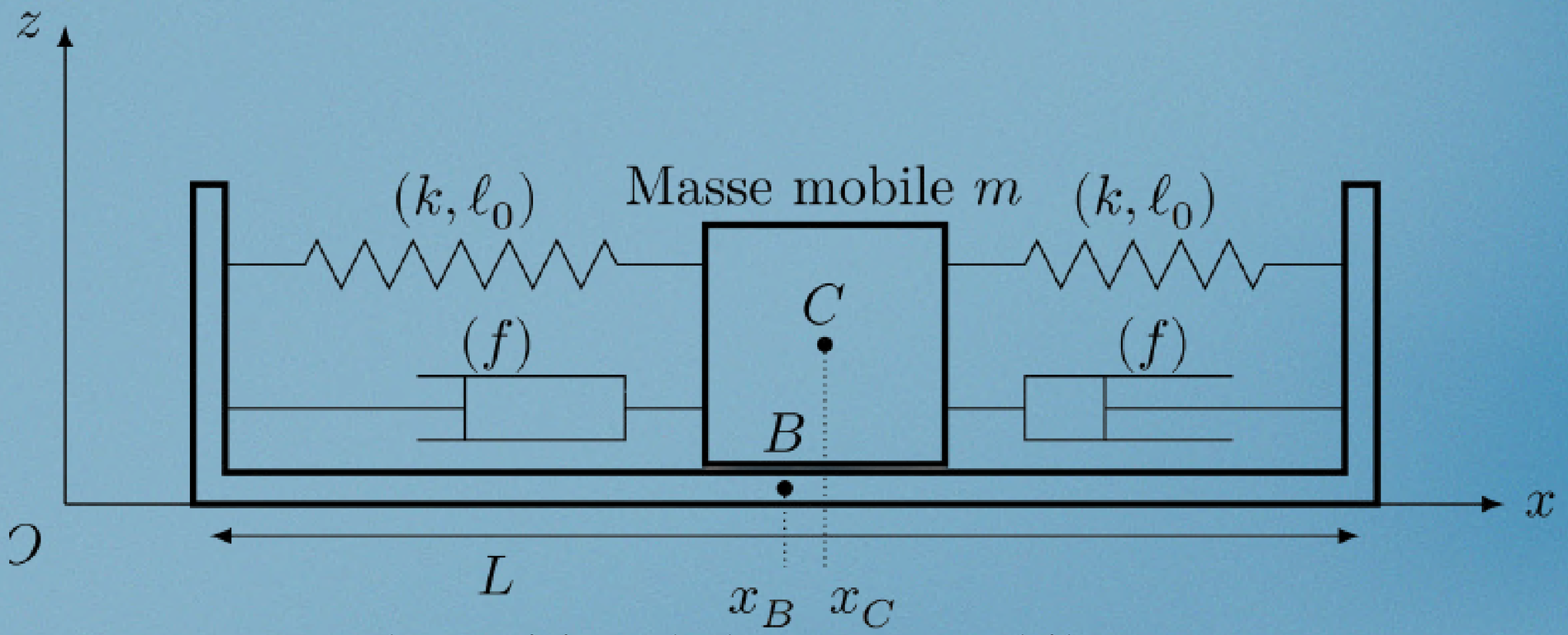
$C_3 - C_4 > 0$

Source : editions-ellipses.fr





Comment fonctionne l'accéléromètre ?

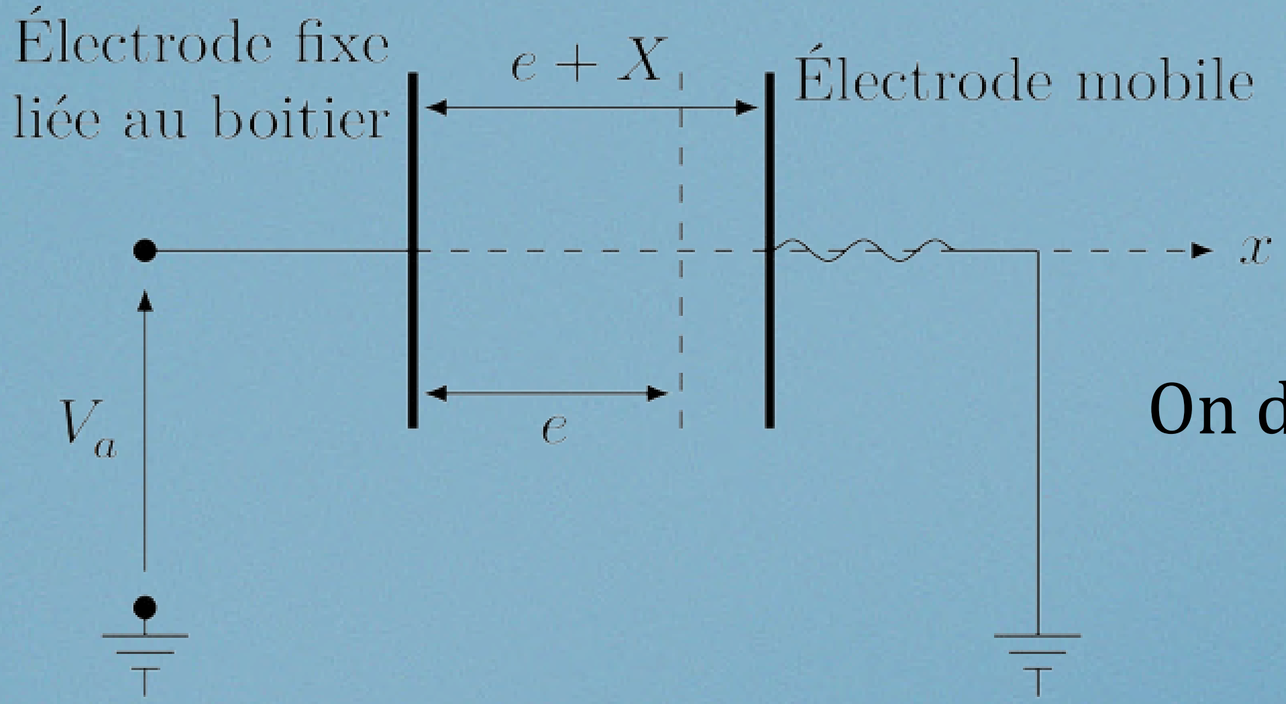


On note $X = x_c - x_b$, la position de la masse mobile par rapport au centre du boitier

$$\ddot{X} + \frac{\omega_0}{Q}\dot{X} + \omega_0^2 X = -a \Longrightarrow X(t) = -\frac{m}{2k} \cdot a(t)$$

Sous réserve d'être dans un certain domaine de fréquence

Comment fonctionne l'accéléromètre ?



$$C = C_0 \cdot \frac{e}{e+X}$$

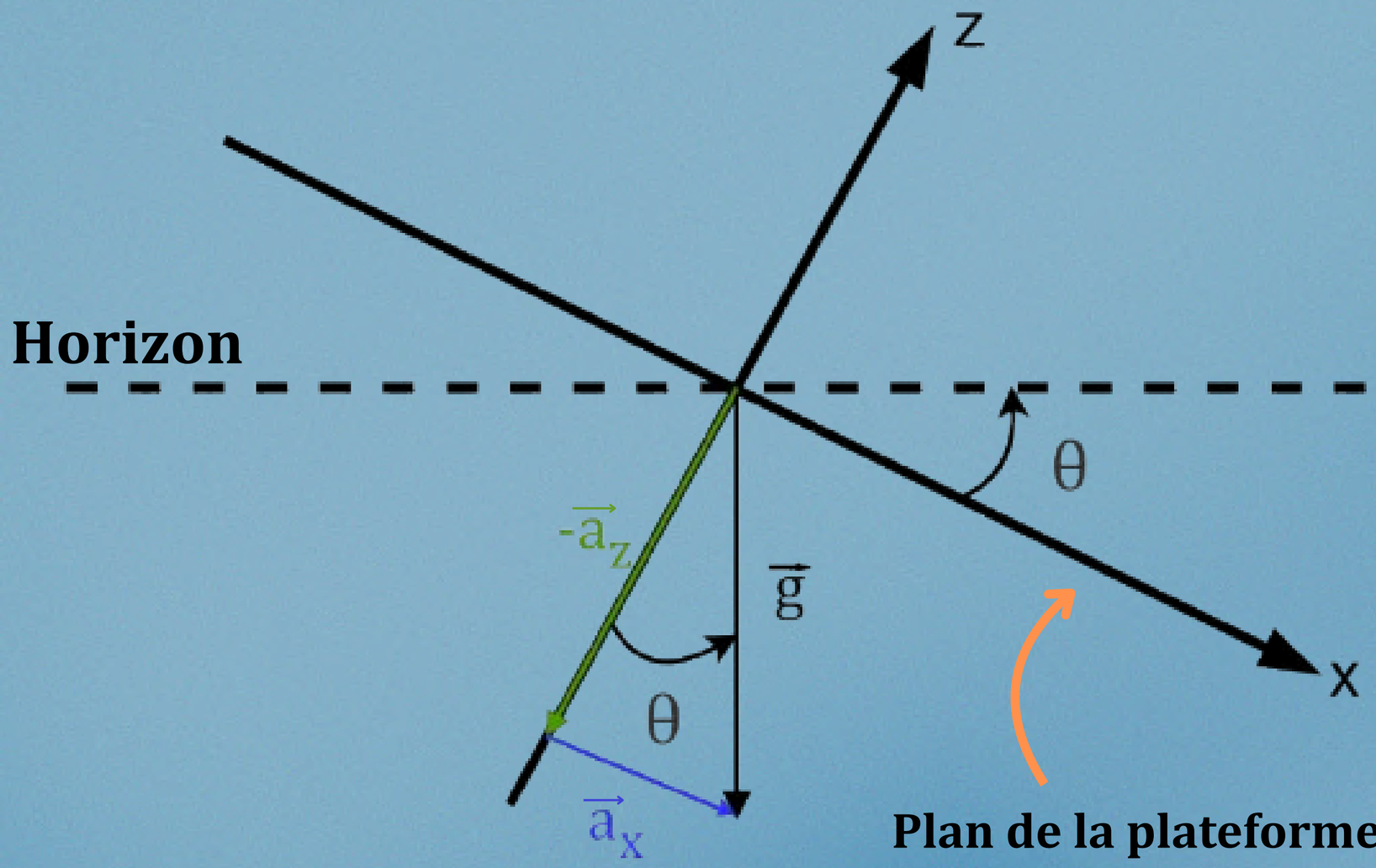
On déduit des relations précédentes que :

$$a = -2ke \cdot \frac{C_0 - C}{m}$$

- C : Capacité
- C_0 : Capacité à X = 0m
- k : Raideur du ressort
- e : Distance entre 2 électrodes

On retrouve donc l'accélération linéaire sur un axe

Pourquoi l'accélération ?



$\vec{g}$: accélération de la pesanteur

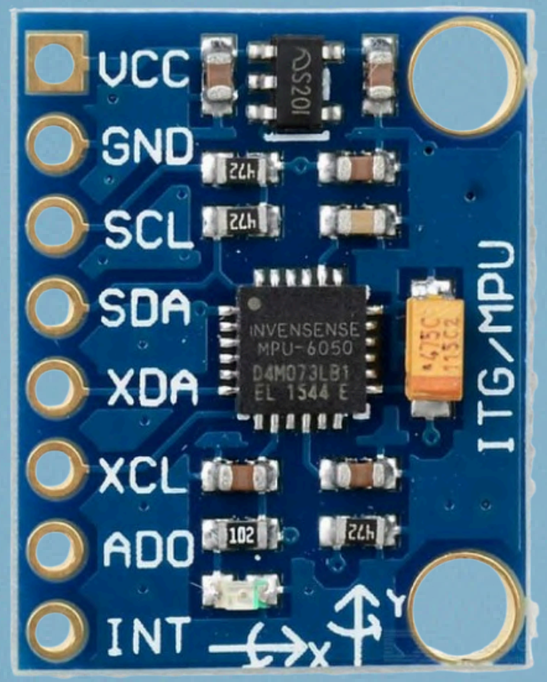
$$\vec{a_x} = g \cdot cos(\theta)\vec{u_x}$$

$$\vec{a_z} = -g \cdot sin(\theta)\vec{u_z}$$

$$\theta = \arctan(-\frac{a_x}{a_z})$$

Plan de la plateforme à un instant t

Filtrage complémentaire



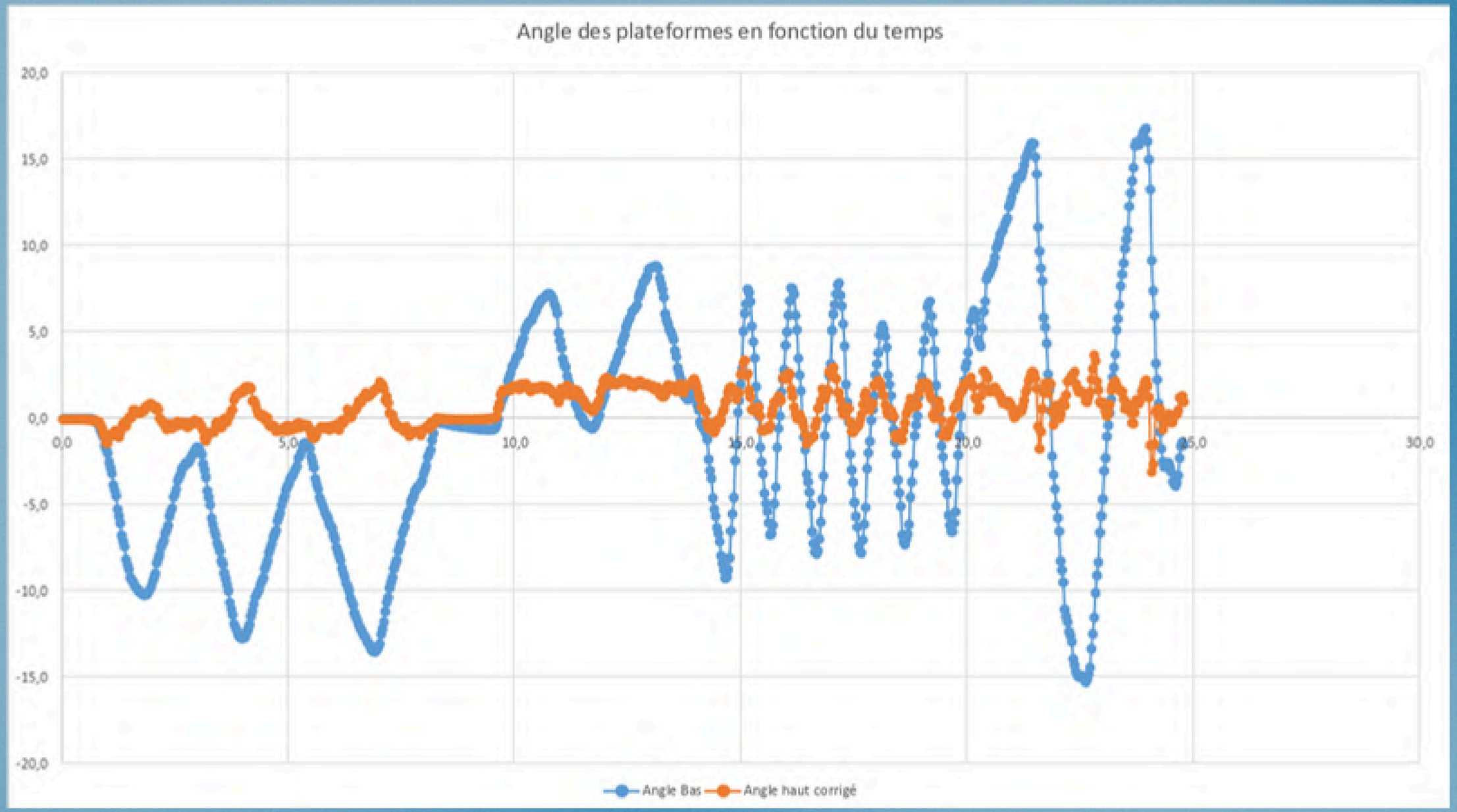
$$\theta(t) = \beta \cdot \theta_{gyro}(t) + (1 - \beta) \cdot \theta_{acc}(t)$$

$$\theta_{gyro}(t) = \int \omega \cdot \mathrm{dt}$$

Avec $\beta = 0.98$ il y a très peu de latence, mais la dérive se voit un peu

II- Mise en œuvre :

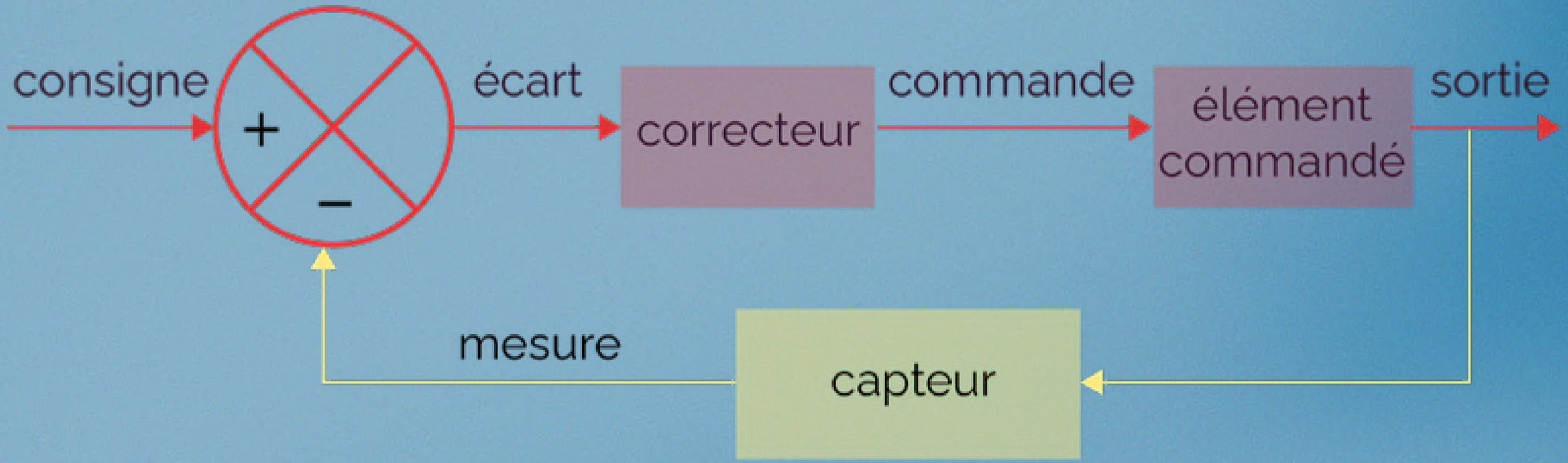
Boucle ouverte



Boucle ouverte



Boucle fermée



Notre consigne : toujours à 0°

Boucle fermée

Réglage du <u>correcteur</u> :

$$C(p) = K + \frac{1}{\tau_i\, p} + \tau_d\, p$$

Méthode de Ziegler-Nichols :

Correcteur	P	PI	PID
K	$0{,}5\ K_{lim}$	$0{,}45\ K_{lim}$	$0{,}6\ K_{lim}$
τ_i	∞	$0{,}83\ T_{osc}$	$0{,}5\ T_{osc}$
τ_d	0	0	$0{,}125\ T_{osc}$

K_{lim} : Gain pour lequel on commence à avoir des oscillations

T_{osc} : Période de ces oscillations

Merci pour votre écoute !

Avez-vous des questions ?



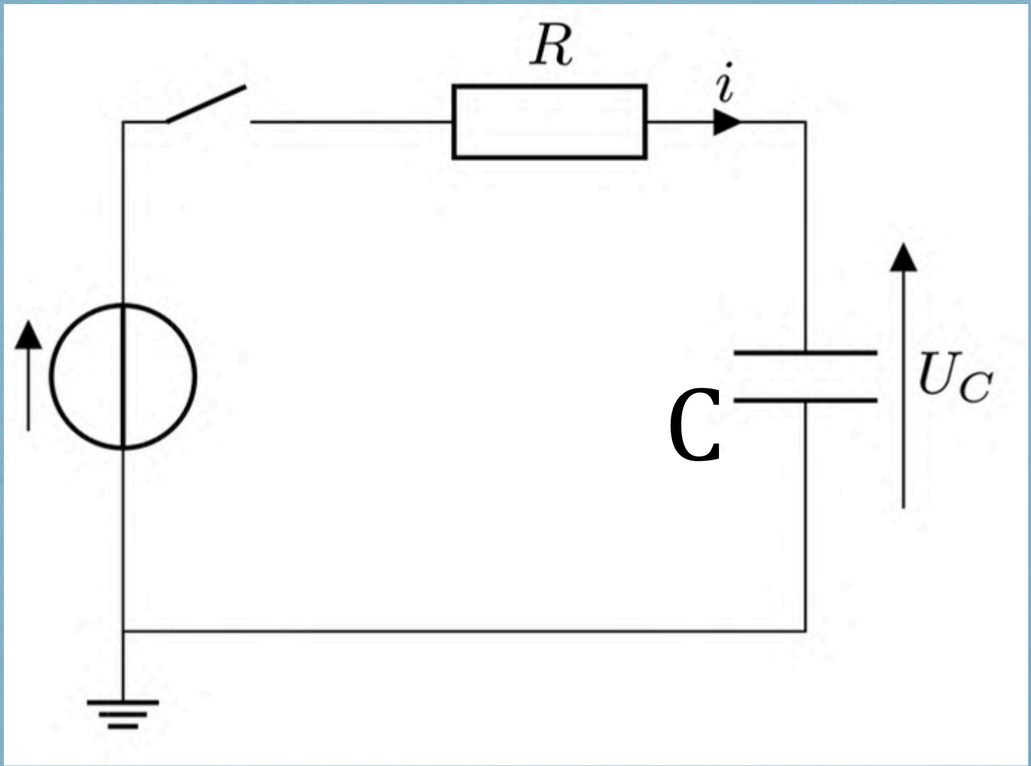
Annexes

Annexe : Cycles, Boucles

Cycles : La houle qui doit être compensée revient de manière cyclique

Boucles : Grâce à un asservissement en boucle fermée, le système compense les oscillations subies pour une stabilité de la plateforme supérieure.

Annexe : Mesure de capacité



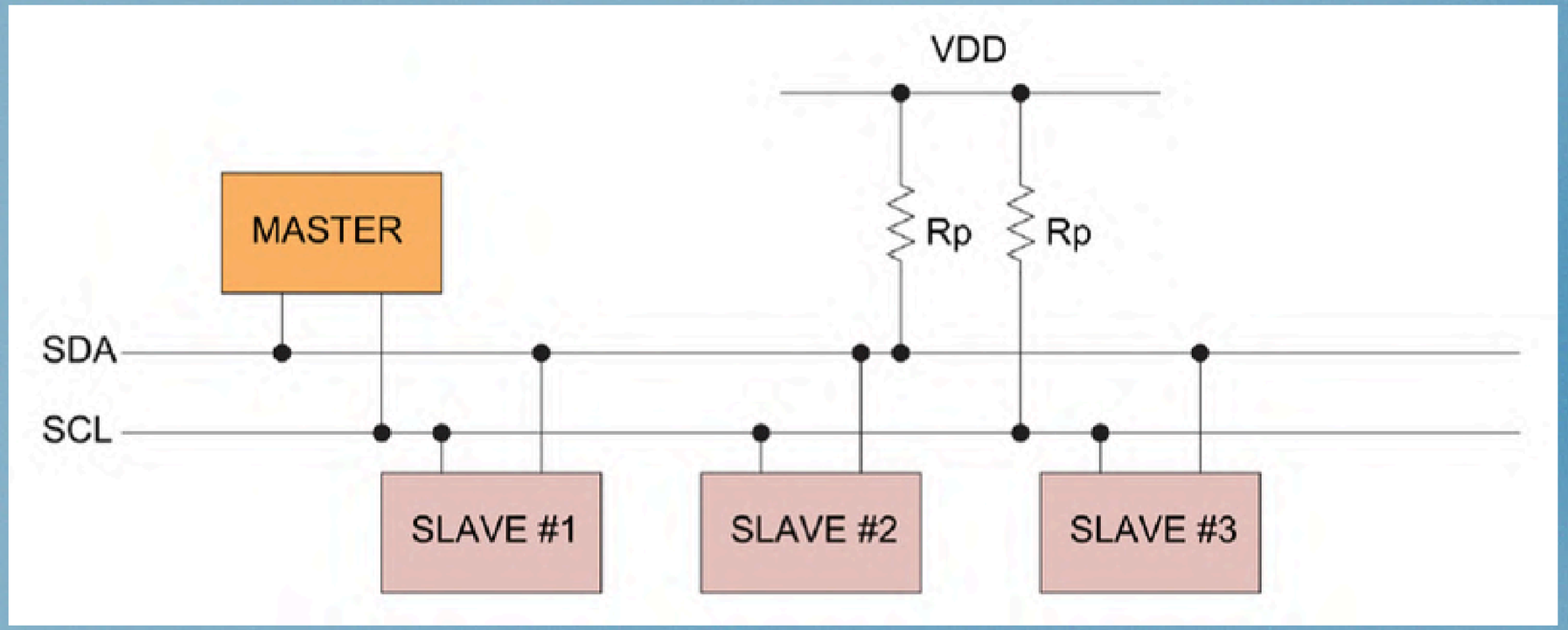
$$U_c(t) = E(1 - e^{-\frac{t}{\tau}})$$

Avec : $$\tau = \frac{1}{RC} \Leftrightarrow C = \frac{1}{R\tau}$$

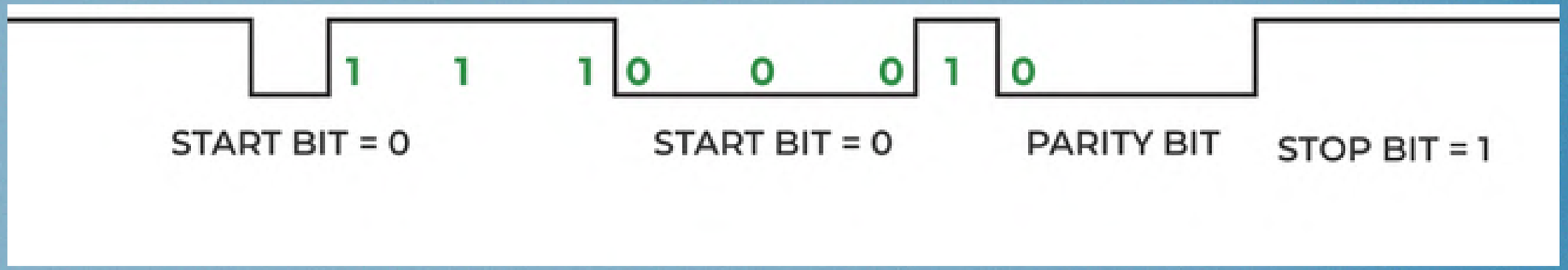
De plus :

$$U_c(\tau) = 0.63 \cdot E$$

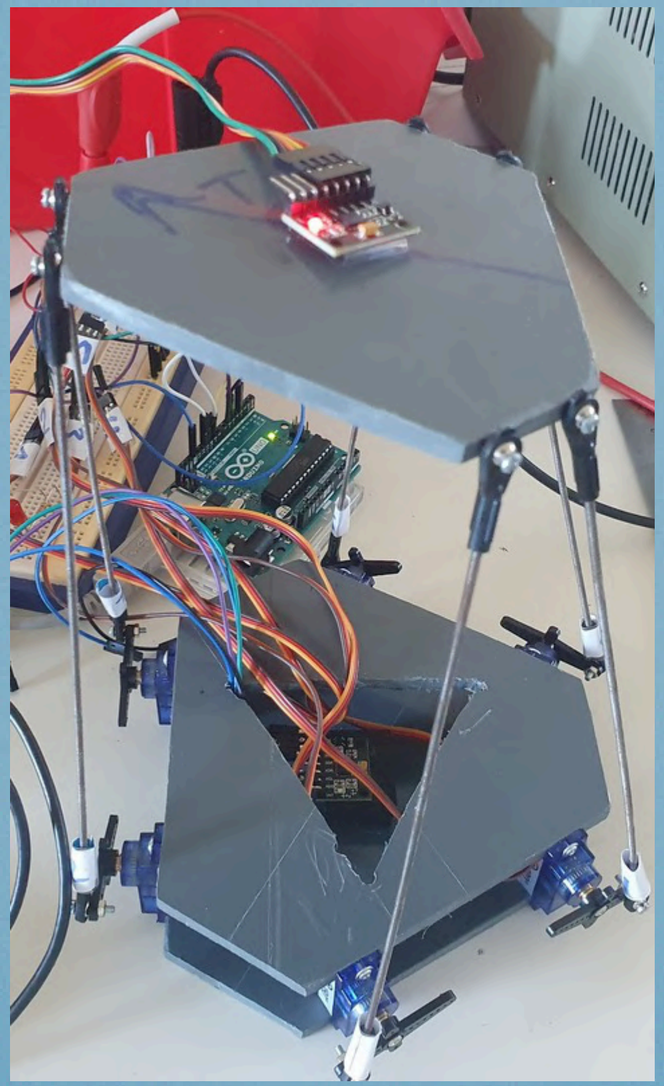
Annexe : La transmission I^2C

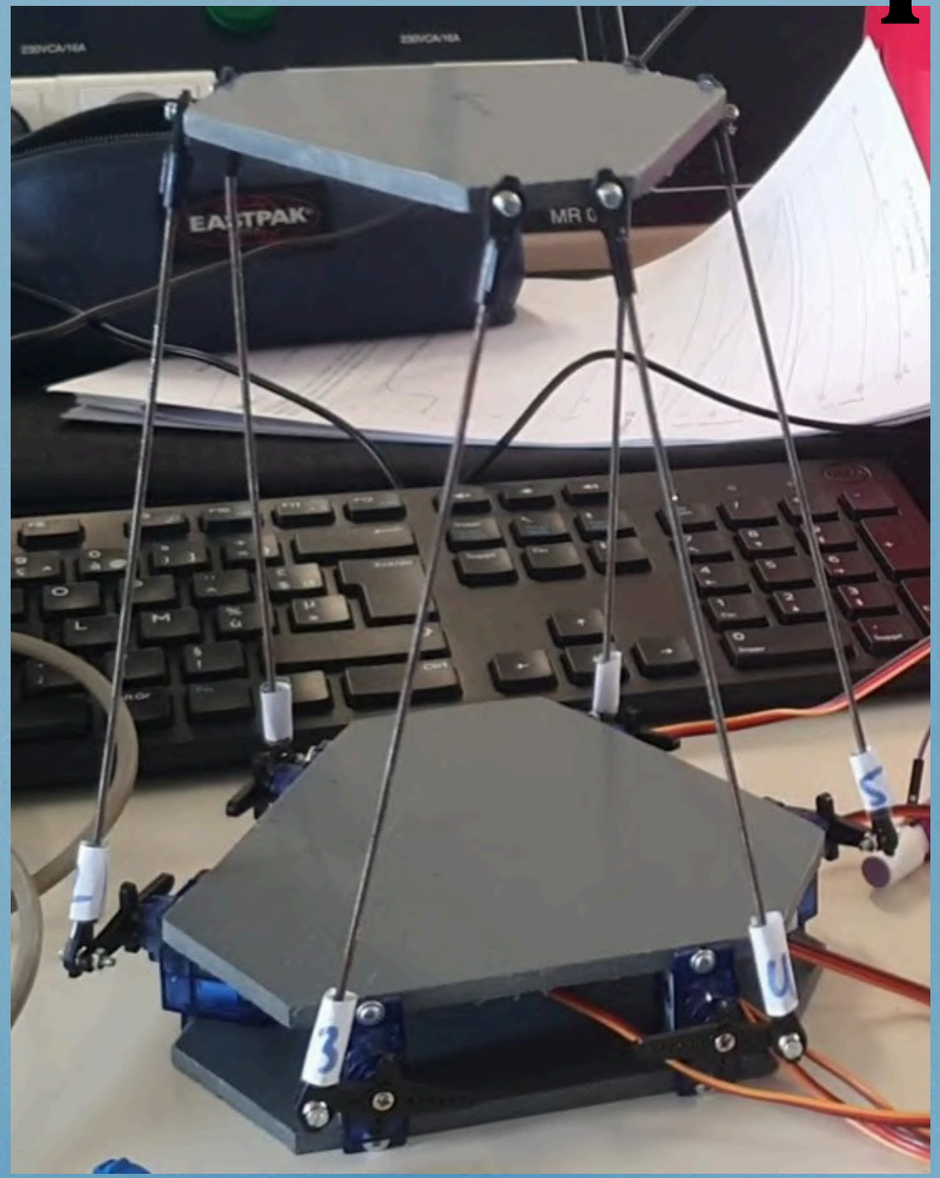


Annexe : La Transmission UART



Annexe : La maquette







Annexe : La maquette



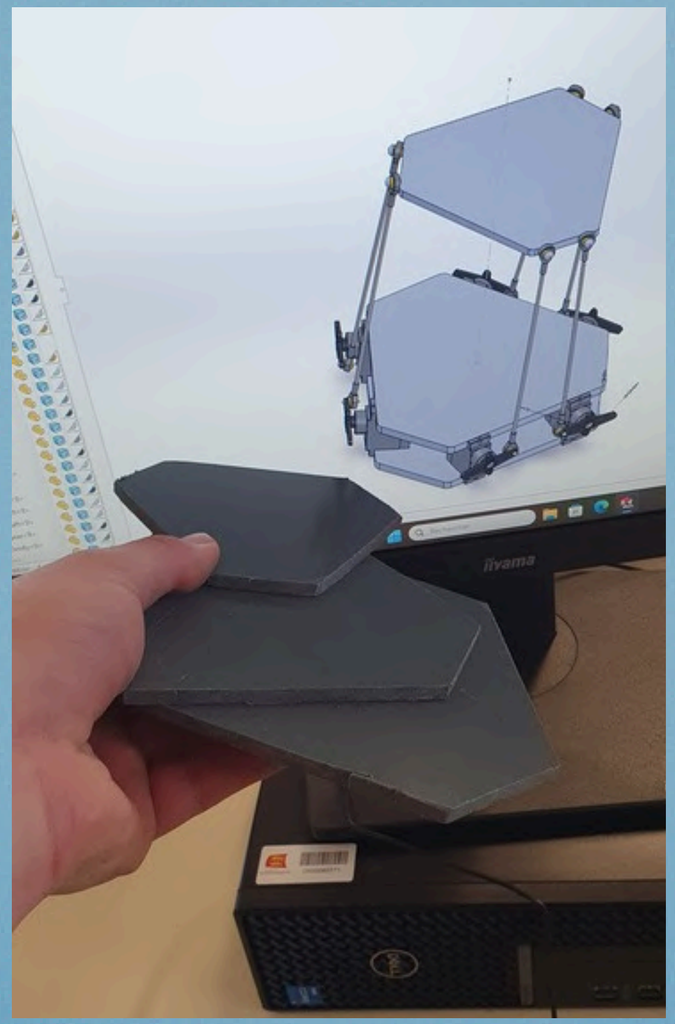


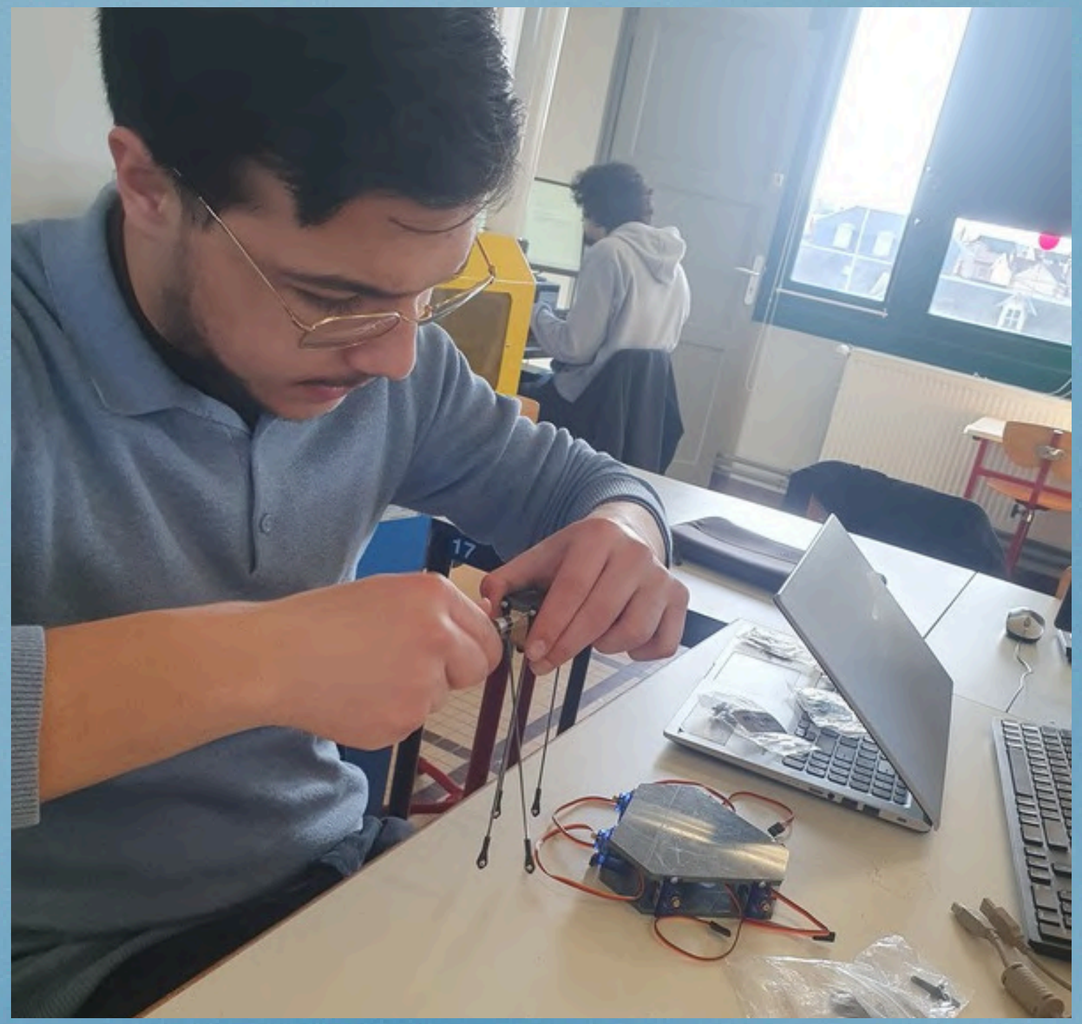
Annexe : La maquette





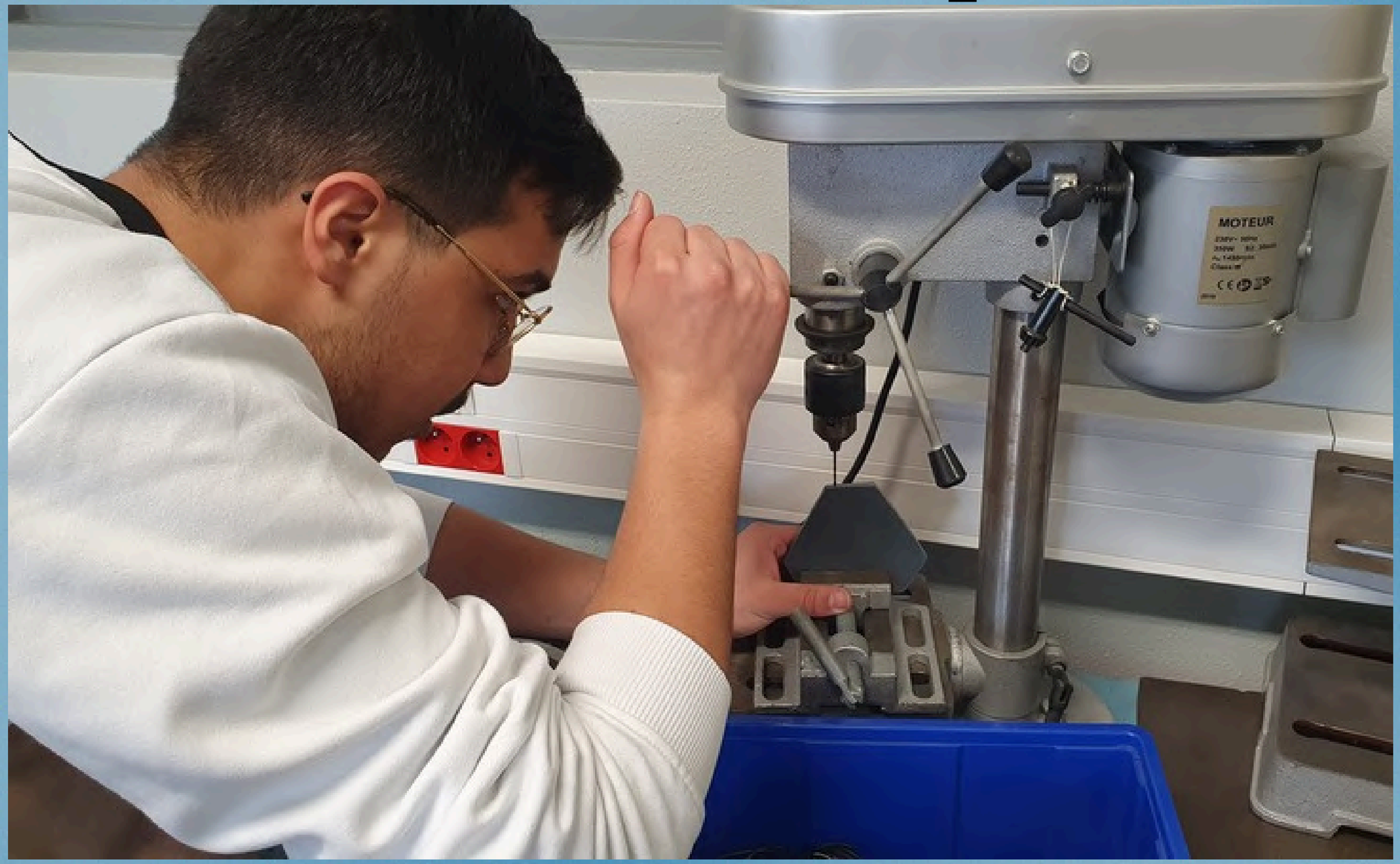
Annexe : La maquette







Annexe : La maquette



Annexe : Régression

```python
import numpy as np
import matplotlib.pyplot as plt

#Données
x = np.array([0, 5, 10, 15, 20, 25, 30, 35, 40, 45, 50, 55])
y = np.array([0.4, 2.6, 4.5, 5.9, 7.0, 8.33, 9.4, 10.7, 11.7, 12.6, 13.45, 14.25])

#Régression
a1, b1      = np.polyfit(x, y, 1)
a2, b2, c2 = np.polyfit(x, y, 2)


#Mise en forme
x_line = np.linspace(0, 55, 300)

fig, ax = plt.subplots(figsize=(9, 5))


#Traçage des courbes
ax.scatter(x, y, color='steelblue', s=60, zorder=5, label='Mesures')
ax.plot(x_line, np.polyval([a1, b1],     x_line), color='tomato',   linewidth=2,
linestyle='--', label='Linéaire')
ax.plot(x_line, np.polyval([a2, b2, c2], x_line), color='seagreen', linewidth=2,
label='Quadratique')
```

Annexe : Régression

```
23
24 ax.text(0.03, 0.97, f"$y = {a1:.4f}x + {b1:.4f}$",
   transform=ax.transAxes, fontsize=15, color='tomato',   va='top')
25 ax.text(0.03, 0.87, f"$y = {a2:.5f}x^2 + {b2:.4f}x + {c2:.4f}$",
   transform=ax.transAxes, fontsize=15, color='seagreen', va='top')
26
27
28 #Formalisation
29 ax.set_xlabel("Amplitude servo (°)")
30 ax.set_ylabel("Angle plateforme (°)")
31 ax.set_title("Régression linéaire et quadratique")
32 ax.legend(loc='lower right')
33 ax.grid(True, linestyle=':', alpha=0.5)
34 ax.set_xlim(0, 55)
35
36
37
38 plt.tight_layout()
39 plt.savefig("regression.png", dpi=150)
40 plt.show()
```

Annexe : Mesure angulaire

```
1   #include <Wire.h>
2   #include <MPU6050.h>
3
4
5   MPU6050 mpu;
6
7   float angle_roulis  = 0.0;
8   float angle_tangage = 0.0;
9   unsigned long temps_precedent = 0;
10  const float beta = 0.98;
11
12  const float ACCEL_SCALE = 16384.0;
13  const float GYRO_SCALE  = 131.0;
14
15  void setup() {
16    Serial.begin(115200);
17    Wire.begin(8, 9);
18    mpu.initialize();
19    temps_precedent = millis();
20
21  }
```

Annexe : Mesure angulaire

```
23  void loop() {
24    // --- Acquisition---
25    int16_t ax, ay, az, gx, gy, gz;
26    mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
27
28    // --- Accéléromètre ---
29    float axg = ax / ACCEL_SCALE;
30    float ayg = ay / ACCEL_SCALE;
31    float azg = az / ACCEL_SCALE;
32
33    float angle_accel_roulis  = atan2(ayg, sqrt(axg * axg + azg * azg)) * 180.0 / PI;
34    float angle_accel_tangage = atan2(-axg, sqrt(ayg * ayg + azg * azg)) * 180.0 / PI;
35
36    // --- Gyroscope ---
37    float wx = gx / GYRO_SCALE;
38    float wy = gy / GYRO_SCALE;
39
40    unsigned long maintenant = millis();
41    float dt = (maintenant - temps_precedent) / 1000.0;
42    temps_precedent = maintenant;
43
```

Annexe : Mesure angulaire

```
44    // --- Filtrage complémentaire ---
45    angle_roulis  = beta * (angle_roulis  + wx * dt) + (1 - beta) * angle_accel_roulis;
46    angle_tangage = beta * (angle_tangage + wy * dt) + (1 - beta) * angle_accel_tangage;
47
48
49    Serial.print(angle_roulis);
50    Serial.print(",");
51    Serial.println(angle_tangage);
52
53
54    delay(20);
55  }
```

Annexe : Correction en boucle ouverte

```
#include <Wire.h>
#include <MPU6050.h>
#include <Servo.h>
#include <math.h>

Servo servo1;
Servo servo2;
Servo servo3;
Servo servo4;
Servo servo5;
Servo servo6;

MPU6050 mpu;

float angle_roulis  = 0.0;
float angle_tangage = 0.0;
unsigned long temps_precedent = 0;
const float beta = 0.98;

const float ACCEL_SCALE = 16384.0;
const float GYRO_SCALE  = 131.0;
```

Annexe : Correction en boucle ouverte

```
22
23  void setup() {
24    Serial.begin(115200);
25    Wire.begin();
26    mpu.initialize();
27    temps_precedent = millis();
28
29    servo1.attach(3);
30    servo2.attach(5);
31    servo3.attach(6);
32    servo4.attach(9);
33    servo5.attach(10);
34    servo6.attach(11);
35  }
```

Annexe : Correction en boucle ouverte

```
37  float calculerAmplitudeServo(float angle_plateau) {
38
39      float y = fabs(angle_plateau);
40
41      // Sécurité physique :Saturation imposée
42      if (y > 13.43) y = 13.43;
43
44      // Modèle inverse de la régression
45      float a = -0.00209;
46      float b = 0.3581;
47      float c = 0.7560 - y;
48
49      float delta = (b * b) - (4 * a * c);
50
51      if (delta >= 0) {
52          float x = (-b + sqrt(delta)) / (2 * a);
53
54          //X reste dans [0, 50]
55          if (x < 0) x = 0;
56          if (x > 50) x = 50;
57
58          return x;
59      }
60      return 0.0;
61  }
```

Annexe : Correction en boucle ouverte

```
63  void loop() {
64    // --- Acquisition---
65    int16_t ax, ay, az, gx, gy, gz;
66    mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
67
68    // --- Accéléromètre ---
69    float axg = ax / ACCEL_SCALE;
70    float ayg = ay / ACCEL_SCALE;
71    float azg = az / ACCEL_SCALE;
72
73    float angle_accel_roulis  = atan2(ayg, sqrt(axg * axg + azg * azg)) * 180.0 / PI;
74    float angle_accel_tangage = atan2(-axg, sqrt(ayg * ayg + azg * azg)) * 180.0 / PI;
75
76    // --- Gyroscope ---
77    float wx = gx / GYRO_SCALE;
78    float wy = gy / GYRO_SCALE;
79
80    unsigned long maintenant = millis();
81    float dt = (maintenant - temps_precedent) / 1000.0;
82    temps_precedent = maintenant;
83
84    // --- Filtrage complémentaire ---
85    angle_roulis  = beta * (angle_roulis  + wx * dt) + (1 - beta) * angle_accel_roulis;
86    angle_tangage = beta * (angle_tangage + wy * dt) + (1 - beta) * angle_accel_tangage;
87
```

Annexe : Correction en boucle ouverte

```
88      // ---Correction---
89      float amplitude_absolue = calculerAmplitudeServo(angle_pitch);
90
91      // Détermination du sens de la compensation
92      float sens_compensation = (angle_pitch > 0) ? 1.0 : -1.0;
93
94      // Calcul de la commande
95      int commande_servo = round(amplitude_absolue * sens_compensation);
96
97      RotationX(commande_servo);
98
99      Serial.print(" | Commande Servo: ");
100     Serial.println(commande_servo);
101
102     delay(20);
103   }
104   //---------------------------------//
105   // FONCTIONS FONDAMENTALES
106   //---------------------------------//
107
108   void TournerTrigo1(int ampli) {
109     servo1.write(90 - ampli);
110   }
111   void TournerHoraire1(int ampli) {
112     servo1.write(90 + ampli);
113   }
```

Annexe : Correction en boucle ouverte

```
void RotationX(int ampli) {
  TournerHoraire1(ampli);
  TournerTrigo2(ampli);
  TournerTrigo3(ampli);
  TournerHoraire4(ampli);
  TournerTrigo5(ampli);
  TournerHoraire6(ampli);
}

void RotationY(int ampli) {
  TournerHoraire1(ampli);
  TournerHoraire2(ampli);
  TournerHoraire6(ampli);
  TournerTrigo3(ampli);
  TournerTrigo4(ampli);
  TournerTrigo5(ampli);
}
```

Annexe : Calcul mathématiques

$$y = ax^2 + bx + c$$

$$y = a\left(x + \frac{b}{2a}\right)^2 - \frac{b^2}{4a} + c$$

$$\left(x + \frac{b}{2a}\right)^2 = \frac{4a \cdot y + b^2 - 4ac}{4a^2}$$

Annexe : Calcul mathématiques

$$x + \frac{b}{2a} = \pm\frac{\sqrt{b^2 - 4a(c - y)}}{2a}$$

$$x = \frac{-b \pm \sqrt{b^2 - 4ac + 4ay}}{2a}$$